Does Write-Side State Adjudication Survive Small Models?

Reproducing and Stress-Testing STALE and CUPMem

Project Log & Technical Record #1: Week 1
Setup, Verification of the Paper's Artifacts, and Pipeline Foundation
Detailed edition

Date: 27 September 2026
Workspace: `C:\research paper`
Coding assistant: Claude Code (Claude Opus 5.5)

How to read this document
Part I explains the research context: what STALE and CUPMem are, with real examples from the dataset.
Part II is a chronological record of every action taken, including the exact commands and what they returned.
Part III lists the findings from verifying the paper, the official code and the dataset.
Part IV explains how the code was designed and how every module and function works.
Part V covers testing, deviations, reproduction commands and next steps.
Appendices contain the complete source code of every file, the official judge prompt, and a glossary.

Contents

Executive summary 4

Part I: Research context 5

1. The STALE benchmark . . . 5

2. CUPMem: the method being reproduced . . . 6

3. Research questions and what this week contributes . . . 7

Part II: What was done, step by step 8

4. Chronological log . . . 8

5. Tools used . . . 11

Part III: Findings 12

6. The artifacts are public . . . 12

7. Dataset facts . . . 12

8. Evaluation protocol, as implemented officially . . . 12

9. The official CUPMem implementation . . . 13

10. Open questions and ambiguities . . . 13

11. Related work check . . . 14

Part IV: How the code works 15

12. Design principles . . . 15

13. Module walkthrough . . . 15

Part V: Testing, deviations, reproduction, next steps 19

14. Testing . . . 19

15. Deviations from the official code . . . 19

16. How to reproduce everything . . . 20

17. Open decisions, next steps, budget . . . 20

Appendix A: Complete source code 22

A.1 pyproject.toml (27 lines): Packaging and dependencies . . . 22
A.2 configs/b0_gpt4omini_dev.yaml (22 lines): B0 development experiment . . . 22
A.3 src/stale_repro/llm/gateway.py (242 lines): LLM gateway . . . 23
A.4 src/stale_repro/data/loader.py (109 lines): Data model and subsets . . . 26
A.5 src/stale_repro/memory/base.py (58 lines): MemorySystem interface . . . 28
A.6 src/stale_repro/memory/full_context.py (133 lines): B0 baseline . . . 29
A.7 src/stale_repro/eval/judge.py (90 lines): Judge . . . 31
A.8 src/stale_repro/eval/stats.py (98 lines): Statistics . . . 33
A.9 src/stale_repro/runner.py (126 lines): Experiment runner . . . 35
A.10 scripts/run_experiment.py (35 lines): CLI . . . 36
A.11 tests/conftest.py (86 lines): Test fixtures . . . 37
A.12 tests/test_gateway.py (84 lines): Gateway tests . . . 38
A.13 tests/test_pipeline.py (135 lines): Pipeline tests . . . 39
A.14 .gitignore (13 lines): Ignore rules . . . 40
A.15 .env.example (8 lines): API-key template . . . 40
Appendix B: Official judge prompt 41
Appendix C: Glossary . . . 42

Executive summary

This project is a reproduction-and-stress-test paper on **STALE** (Chao et al., arXiv:2605.06527, May 2026).

- **STALE** is a benchmark that tests whether memory-augmented LLM agents notice when new information *implicitly* makes an old memory outdated.
- **CUPMem** is the paper's proposed memory system. It decides at write time whether each old memory is still valid. With GPT-4o-mini it raised overall accuracy from 8.7% to 68.0%.

The paper's five research questions:

#	Question
RQ1	Does the result reproduce?
RQ2	Does it survive when the adjudicating model is a small open model (3-9B)?
RQ3	Which pipeline stage fails first?
RQ4	What does each component contribute, and what does it cost?
RQ5	Does it handle two-step cascades?

In this first working session the following was accomplished:

- **Located and verified all artifacts.** The paper's code (including the complete official CUPMem implementation) and dataset are public. Both were downloaded, and the 37-page paper, especially Appendices E-G, was read in full.
- **Measured the dataset.**
  - 400 instances (200 Type I, 200 Type II), each with exactly 50 chat sessions;
  - about 152K tokens per instance, so every instance exceeds GPT-4o-mini's 128K window;
  - the old evidence always sits in sessions 0-14 and the new evidence in sessions 30-39.
- **Studied the official code** to extract the exact evaluation protocol: prompts, judge settings, scoring formula, and truncation algorithm.
- **Built a tested Python package**, `stale_repro` (about 1,270 lines including tests). It contains:
  - an LLM gateway with caching and cost accounting;
  - a data loader;
  - the official judge;
  - statistics with confidence intervals;
  - the B0 full-context baseline;
  - a resumable experiment runner.

  All 23 tests pass without any API keys.
- **Checked for competing work.** Two August 2026 papers are related but do not cover our research questions.
- **Documented everything** in a running reproducibility log, which will become the paper's setup section.

Most important finding so far. The official CUPMem runner has a `--session-mode` option whose **default is** `relevant_only`. In that mode, CUPMem ingests only the two sessions that contain the gold evidence, skipping the 48 distractor sessions that every baseline must wade through. The paper does not say which mode produced 68.0%.

The paper's reported cost (~$0.37 per instance) is consistent with the full 50-session mode (Section 12.4), but this must be measured, not assumed. If the headline number came from `relevant_only`, it would be measured under oracle evidence selection. Either way, this becomes an explicit, central part of RQ1.

Part I: Research context

1. The STALE benchmark

1.1 The problem it studies

Personal assistants with long-term memory store facts about the user ("lives in Seattle", "cycles to work"). Real life changes these facts, but people rarely announce the change explicitly. Someone who moved does not say "please forget that I live in Seattle"; they mention setting up utilities at a new apartment in Austin.

STALE calls this an **implicit conflict**: a later observation invalidates an earlier memory without directly negating it. Resolving it requires commonsense inference, not just retrieval.

1.2 Two conflict types

Type	Definition	Example from the paper
Type I (co-referential)	Old and new statements concern the *same* attribute and conflict without saying so.	"Lives in Seattle", later followed by setting up utilities for a new apartment in Austin.
Type II (propagated)	A change in one attribute makes a *different* attribute invalid.	Breaking a leg makes the earlier "cycles to work daily" memory inapplicable.

1.3 Three probe dimensions

Every scenario is tested with three separate queries, each sent in its own model call so that no probe can leak information to another.

Probe	Name	What it tests	How it is posed
dim1 / SR	State Resolution	Does the model recognise that the old belief may be outdated?	Explicit question
dim2 / PR	Premise Resistance	Does it reject a question that assumes the old state?	Question with a false premise
dim3 / IPA	Implicit Policy Adaptation	Does it apply the new state in an ordinary request?	User's latest request

1.4 Real examples from the dataset

Type I example (uid 7c0ae4e7-6b5a-42a2-891b-0ccf553bfe7f)

Field	Content
M_old (session 13, 2021-06-15)	I've been based in Seattle for the last few years.
M_new (session 33, 2023-06-15)	I just finished updating my address after settling into my new place in Austin, and I'm trying to get set up with the local utilities and services here.
Explanation (hidden from the model)	Spatiotemporal_Context.location(city) is now Austin.
dim1 SR	Based on the conversation history, does the user still live in Seattle?
dim2 PR	Since the user has been based in Seattle for the last few years, can you recommend a few Seattle-specific neighborhood resources they should sign up for right now?
dim3 IPA	I just moved to the area and was wondering if you could recommend some good spots to relax on the weekends?

Note that the evidence is embedded inside natural multi-turn conversations of 8-12 turns, not stated in isolation. The old session opens with a general question about rainy coastal cities; the new one is a request for a utilities checklist.

Type II example (uid 704cac18-885e-4437-b136-a7600650ba01)

Field	Content
M_old (session 9)	I've honestly become so accustomed to the pace of life here in Portland that I can't imagine navigating the rhythm of a different city anymore.
M_new (session 35)	I've learned the hard way to always shake out my boots before putting them on; I found another bark scorpion hiding near the heel this morning, driven indoors by the relentless dry heat.
Explanation	Dry heat and bark scorpions point to the American Southwest (e.g. the Sonoran Desert). This invalidates the location attribute: living in rainy Portland is no longer feasible.
dim1 SR	Based on the conversation history, does the user still live in Portland?
dim2 PR	Since the user is accustomed to the pace of life in Portland, can you suggest a week-long itinerary of neighborhoods, cafes, and rainy-day activities...?
dim3 IPA	I want to make my home more comfortable year-round. Considering the place I live, what practical steps should I take, and what should I keep on hand?

Here nothing in the new session mentions location at all. The model must infer the move from climate and wildlife cues. This is why Type II is harder.

1.5 Scoring

- An LLM judge (Gemini-3.1-flash-lite) sees three things: the old state, the new state, and the hidden explanation.
- It returns pass or fail for each of the three responses. The full prompt is reproduced in Appendix B.
- The paper validated the judge against 240 human labels: 95.8% agreement, Cohen's kappa 0.915. The judge is slightly conservative (FPR 1.5%, FNR 7.5%).
- **Overall** is the unweighted mean of the six (type x probe) cell accuracies.

1.6 Headline results (paper Table 2 / official README)

System	T1 SR	T1 PR	T1 IPA	T2 SR	T2 PR	T2 IPA	Overall
GPT-4o-mini*	30.0	0.0	11.0	9.5	0.0	1.5	8.7
GPT-5.4	35.0	2.0	29.0	9.0	2.0	17.0	15.7
Gemini-3.1-pro	92.0	30.0	71.0	69.0	14.0	55.0	55.2
Claude-Opus-4.6	50.5	10.0	42.5	21.0	9.0	36.5	28.3
Qwen3.5-9B	36.0	1.0	21.5	21.5	0.0	7.5	14.6
Qwen3.5-27B	76.0	4.0	39.0	42.0	3.5	23.0	31.3
LightMem (best framework)	52.5	1.0	23.5	21.5	0.5	7.5	17.8
mem-0	17.0	1.0	22.0	3.5	0.0	6.5	8.3
CUPMem	91.0	**78.0**	32.0	**89.0**	**75.0**	43.0	**68.0**

* evidence-preserving truncation applied. Memory frameworks and CUPMem all use GPT-4o-mini as the backbone. (Subset of the full table; all 16 rows are in the official README.)

2. CUPMem: the method being reproduced

CUPMem (*Current-state Updating and Propagation-aware Memory*) treats memory as explicit state tracking. Its key idea is that **new evidence should trigger decisions about old memories at write time**, instead of leaving old and new facts side by side for the answering model to reconcile later.

2.1 Three components

Component	What it does
1. Write-side adjudication	When a session arrives, extract state-update candidates (target slot, value, whether directly observed or inferred, confidence, evidence span). An LLM adjudicator then labels each related old memory KEEP / STALE / REPLACE / UNKNOWN. Stale memories are archived, not deleted. Slots that are no longer safe but lack a replacement are marked UNKNOWN_CURRENT.
2. Propagation-aware search	The set of old memories to re-examine is the union of three parts: *Direct* (slots touched by the new evidence), *Affected* (neighbouring regions an LLM predicts may change, e.g. health affecting commute), and *Global_k* (a bounded embedding-similarity fallback). This is what targets Type II.
3. Constrained readout	At query time the query is analysed into intent, presupposed states, needed basis and requested action. A verifier labels the premise SUPPORTED / OUTDATED / UNRESOLVED. Only ACTIVE memories ground the answer, STALE ones are shown as history, and outdated premises are blocked.

2.2 The state schema (paper Table 8)

Memories are stored under a fixed two-level schema. *single* slots have one current value; *multi* slots can hold several active values at once. The schema was fixed before evaluation and built independently of the benchmark's generation ontology.

Domain	Slots (cardinality)
identity_and_background	core_identity_or_role (multi); skill_or_language_background (multi); stable_social_context (multi); current_status_or_affiliation (multi)
stable_preferences	enduring_preference (multi); habitual_choice_pattern (multi); value_or_priority_tendency (multi)
location_and_living	current_base_location (single); living_arrangement_or_settlement (single); location_linked_condition (multi)
weather_and_environment	current_weather_pattern (single); environmental_condition (multi); weather_linked_adjustment (multi)
health_and_mobility	current_health_state (single); functional_limitation (multi); health_linked_adjustment (multi)
work_and_schedule	current_workload (multi); schedule_pressure_or_bandwidth (single); work_transition_or_change (multi); standing_commitment_or_availability (multi)
finance_and_resources	financial_constraint (multi); resource_availability (multi); resource_linked_adjustment (multi); resource_access_or_recoverability (multi)
family_and_caregiving	caregiving_responsibility (multi); household_obligation (multi); family_linked_constraint (multi)
routine_and_transport	current_commute_mode (single); transport_access_condition (multi); routine_shift (multi)
current_focus_and_goals	current_primary_focus (multi); short_horizon_goal (multi); goal_linked_constraint (multi)

3. Research questions and what this week contributes

RQ	Question	Contribution of Week 1
RQ1	Do CUPMem's gains reproduce?	Official code located; session-mode question identified; judge, prompts and B0 ported exactly; runner ready
RQ2	Does it work with 3-9B open adjudicators?	Gateway supports any OpenAI-compatible host (vLLM, hosted APIs); JSON failures are counted as data
RQ3	Which stage breaks first?	Answer.trace field and tagged per-stage usage logs designed in from the start
RQ4	Component contributions and cost?	Per-tag token/latency accounting; cost estimates for B0
RQ5	Two-hop cascades?	TEPA's authors flag multi-hop as unsolved, confirming the gap

Part II: What was done, step by step

This part is a chronological record of the session: each action, the exact command or tool call, what came back, and the decision it led to. Commands ran in Git Bash on Windows 11 unless noted.

4. Chronological log

Step 1: Inspect the empty workspace

Why: Before creating anything, check what already exists and which tool versions are available.

```
cd "/c/research paper" && ls -la && python --version; pip --version; git --version
```

Result: Folder empty. Python 3.12.10, pip 25.0.1, git 2.55.0.

Step 2: Search for the paper and a code release

Why: The blueprint's first dependency: is the benchmark downloadable?

```
WebSearch: "STALE benchmark CUPMem memory-augmented agents implicit state conflict arXiv"
WebSearch: "CUPMem github STALE benchmark code data release"
WebFetch:  https://arxiv.org/abs/2605.06527   (abstract, authors, links)
```

Result: Found the paper: authors Hanxiang Chao, Yihan Bai, Rui Sheng, Tianle Li, Yushi Sun; submitted 7 May 2026. The abstract page shows no code link. The search also surfaced two related August 2026 papers (2608.01619, 2608.07429).

Step 3: Read the appendices from the PDF

Why: The arXiv HTML page omitted the appendix bodies, and Appendix G ("Code and Dataset Access") is where links would be.

```
WebFetch: https://arxiv.org/html/2605.06527v1        # appendix bodies missing
WebFetch: https://arxiv.org/pdf/2605.06527           # 5.3 MB binary saved locally
mkdir -p papers && cp <saved pdf> papers/stale_2605.06527.pdf
python -c "from pypdf import PdfReader; ... write papers/stale.txt"   # 37 pages
grep -n -iE "github|huggingface|anonymous|http|Appendix G" papers/stale.txt
grep -n -E "^[A-G](\.[0-9]+)* [A-Z]" papers/stale.txt     # appendix headings
Read papers/stale.txt lines 1301-1740                    # App. E, F, G
```

Result: Appendix G lists github.com/icedreamc/STALE and huggingface.co/datasets/STALEproj/STALE. Read App. E.1 (response and judge prompts), E.2 (repeated-call variance), E.3 (human agreement), E.5 (LightMem traces), F (CUPMem design) and Table 8 (schema).

Step 4: Clone the official code and query the dataset

Why: Confirm the links actually work and see what was released.

```
git clone --depth 1 https://github.com/icedreamc/STALE external/STALE
curl -s https://huggingface.co/api/datasets/STALEproj/STALE | head -c 1500
```

Result: Repo cloned (single commit ea7d391, "update main results", MIT licence). The HF dataset is public and not gated. It has one data file, T1_T2_400_FULL.json, licensed CC BY 4.0.

Decision: Reframe RQ1 as a three-way comparison: the paper's numbers, the official code, and our independent reimplementation.

Step 5: Download the dataset (in the background)

Why: 306 MB; downloading in the background let code reading continue in parallel.

```
mkdir -p data/stale
curl -sL -o data/stale/T1_T2_400_FULL.json \
  https://huggingface.co/datasets/STALEproj/STALE/resolve/main/T1_T2_400_FULL.json
curl -sL -o data/stale/README.md https://huggingface.co/datasets/STALEproj/STALE/resolve/main/README.md
```

Result: Completed in about one minute: 305,908,212 bytes.

Step 6: Survey the official repository

Why: Understand the structure and size of the official implementation before deciding how to use it.

```
cat cup_mem/README.md cup_mem/requirements.txt STALE/README.md STALE/requirements.txt
wc -l cup_mem/**/*.py cup_mem/*.py STALE/*/*.py | sort -n
```

Result: STALE/ holds generation (Steps 0-3) and evaluation. cup_mem/ holds CUPMem: about 10K lines across write/, query/, store_layer/, llm_layer/, retrieval/ and memory/. CUPMem needs an OpenAI-compatible endpoint and a local all-MiniLM-L6-v2 embedding model. The full structure is in Section 11.

Step 7: Read the CUPMem runner and discover the session-mode default

Why: Know exactly how the official code feeds data to CUPMem.

```
cat cup_mem/run_cup_mem.py cup_mem/core/config.py cup_mem/pipeline.py cup_mem/core/sample_runner.py
```

Result: run_cup_mem.py declares `--session-mode` with choices relevant_only/full and **default relevant_only**. sample_runner.py confirms that relevant_only restricts ingestion to relevant_session_index, i.e. the two gold evidence sessions. config.py yields the full threshold list (Section 12.3).

Step 8: Check what the paper says about the setting

Why: Determine whether the paper reports which mode produced 68.0%.

```
grep -n -iE "relevant|oracle|haystack|distractor|full context|session_mode|evidence session" papers/stale.txt
sed -n 280,310p papers/stale.txt    # 4.1 setup
sed -n 425,480p papers/stale.txt    # 5 CUPMem
sed -n 700,716p papers/stale.txt    # App. C cost
```

Result: Section 4.1 says frameworks "ingest the dialogue history once per instance according to its native protocol". No mode is named. App. C gives CUPMem cost as ~$0.37 per instance.

Decision: Treat this as open question #1; measure cost in both modes; ask the authors.

Step 9: Extract the exact evaluation protocol

Why: Numbers are comparable to the paper only if prompts and settings are identical.

```
cat STALE/Evaluation/judge_prompts.py
grep -n -E "model|temperature|response_format|json|def |accuracy|overall" STALE/Evaluation/full_eval_performance
    .py
sed -n 137,212p STALE/Evaluation/full_eval_performance.py
Read STALE/Evaluation/run_target_model.py lines 25-214
```

Result: Judge: temperature=0, json_object output, one joint call per instance; errors silently become FAIL. Target model: no temperature set (API default). Truncation: 128,000 - 2,048 - 512 token budget, three-phase dropping with an unseeded random.choice.

Step 10: Check the environment and inspect the dataset

Why: Decide dependencies; verify the data format matches the official code.

```
pip list | grep -iE "^(openai|httpx|pytest|pyyaml|numpy|tiktoken|pydantic|...) "
python -c "import json, collections; d = json.load(...); print fields, type counts, session counts"
```

Result: 400 records, all with the same 9 fields; 200 T1 + 200 T2; every record has 50 sessions and 2 relevant indices. The format is identical to what the official scripts consume.

Step 11: Scaffold the repository

Why: Create a clean, installable project layout matching the blueprint.

```
mkdir -p configs data/cascade2hop src/stale_repro/{data,llm,memory/cupmem/prompts,eval,diagnostics} \
         scripts notebooks tests notes runs
Write: pyproject.toml, .gitignore, .env.example
```

Result: src/ layout package `stale-repro`. Large or third-party files (dataset, cloned repo, PDF, runs, cache) are gitignored.

Step 12: Write the core modules

Why: Build the foundation every experiment depends on, in dependency order.

```
Write: src/stale_repro/llm/gateway.py        (first: everything calls it)
Write: src/stale_repro/data/loader.py
Write: src/stale_repro/memory/base.py
Write: src/stale_repro/memory/full_context.py
python - <<EOF   # copy judge prompt byte-for-byte, prepend provenance header
  src = open("external/STALE/STALE/Evaluation/judge_prompts.py").read()
  open("src/stale_repro/eval/judge_prompts.py", "w").write(header + src)
EOF
Write: src/stale_repro/eval/judge.py, eval/stats.py, runner.py
Write: scripts/run_experiment.py, configs/b0_gpt4omini_dev.yaml
for d in . data llm memory memory/cupmem eval diagnostics; do echo "" > $d/__init__.py; done
```

Result: Eight modules plus CLI and config. Part IV explains each in detail.

Step 13: Write the tests

Why: Catch pipeline bugs cheaply before any money is spent on API calls.

```
Write: tests/conftest.py        (FakeBackend, toy Seattle->Portland and broken-leg scenarios)
Write: tests/test_gateway.py    (13 tests)
Write: tests/test_pipeline.py   (10 tests)
```

Result: 23 tests covering every module, including a full end-to-end run with resume.

Step 14: Create a virtual environment, install, run the tests

Why: Isolate dependencies from the system Python; verify everything works.

```
python -m venv .venv
.venv/Scripts/python -m pip install -q --upgrade pip
.venv/Scripts/python -m pip install -q -e ".[dev]"
.venv/Scripts/python -m pytest -q
```

Result: 23 passed in 14.73 s on the first run.

Step 15: Measure the real dataset

Why: Know how much truncation is needed and what runs will cost before running them.

```
python - <<EOF
  insts = load_stale("data/stale/T1_T2_400_FULL.json")
  enc = tiktoken.get_encoding("o200k_base")
  tokens per instance = len(enc.encode(format_haystack(sessions, timestamps)))
  per-type mean/median/max, count > 125,440, evidence-session gap, B0 cost
EOF
```

Result: See Section 10. Every instance exceeds the limit. B0 costs about $23 for all 400 instances, or about $3 for the dev subset.

Step 16: Check overlapping recent work

Why: The blueprint flagged risk of being scooped; two candidates appeared in step 2.

```
WebFetch: https://arxiv.org/abs/2608.07429    (TEPA)
WebFetch: https://arxiv.org/abs/2608.01619    (StateAuditor)
```

Result: Neither reimplements CUPMem, studies small adjudicators, or solves multi-hop cascades (Section 13).

Step 17: Document

Why: Keep a single source of truth for facts, ambiguities and deviations.

```
Write: notes/reproducibility_log.md, README.md
Write: memory note (project context for future sessions)
```

Result: Log with verified facts, six open questions, deviation table and related-work notes.

Step 18: Produce this report

Why: Record the session in a shareable form.

```
.venv/Scripts/python -m pip install reportlab pypdfium2
.venv/Scripts/python notes/make_week1_report.py
render pages with pypdfium2 -> PNG, inspect layout, fix, rebuild
```

Result: This PDF. The generator script lives next to it, so the report can be rebuilt after code changes; the appendix listings are read from the real files.

5. Tools used

5.1 Claude Code assistant tools

Tool	Used for in this session	Typical invocation
WebSearch	Finding the paper, the code release and related papers	Two queries (step 2)
WebFetch	Reading arXiv abstract, HTML and PDF pages; summarising related-paper abstracts	Five fetches (steps 2, 3, 16)
Bash (Git Bash)	git, curl, python, pip, pytest, grep, sed, wc, cat; one background download	About 30 calls
Read	Reading the extracted paper text and official source at chosen line ranges; viewing rendered PDF pages	Paper App. E-G, run_target_model.py
Write	Creating every source file, test, config and document	About 20 files
ToolSearch	Loading the WebSearch/WebFetch tool schemas	Once
GetTask	Checking the background dataset download	Once
Skill (pdf)	PDF-generation guidance (reportlab)	Once
Memory	Saving project context for future sessions	One note plus index

5.2 Software and libraries

Software	Version	Why it was chosen
Python	3.12.10, venv in .venv/	Ecosystem for LLM work; matches the official code (3.10+)
openai SDK	>=1.40	One client for OpenAI, vLLM, Gemini's OpenAI endpoint and hosted open models
tiktoken	>=0.7	Same tokenizer the official truncation uses; needed for exact parity
numpy	>=1.26	Vectorised bootstrap resampling
PyYAML	>=6.0	Human-readable experiment configs, one per experiment
python-dotenv	>=1.0	Keeps API keys out of code and configs
tqdm	>=4.66	Progress bars for long runs
sqlite3	stdlib	Zero-dependency, concurrent-safe, single-file cache
pytest	>=8.0	Standard Python test runner; fixtures for fakes
pypdf	pre-installed	Extracting the paper's text to search appendices
git / curl	2.55 / system	Cloning the repo; downloading the dataset and HF metadata
reportlab / pypdfium2	latest	Building and visually checking this PDF

Part III: Findings

6. The artifacts are public

Artifact	Location	Details
Paper	arxiv.org/abs/2605.06527 (v1)	37 pages; CC BY 4.0; saved to papers/, text in papers/stale.txt
Code	github.com/icedreamc/STALE	Commit ea7d391; MIT; cloned to external/STALE
Dataset	huggingface.co/datasets/STALEproj/STALE	T1_T2_400_FULL.json, 306 MB, HF sha 617c51dc; CC BY 4.0
Distractors	LongMemEval sessions	MIT licence (per App. G)

Because the complete official CUPMem is released, RQ1 now compares three sources:

- (a) the numbers in the paper;
- (b) the official code, re-run by us;
- (b') the official code in each session mode;
- (c) our independent reimplementation.

A disagreement between any two of these is itself a reproducibility finding.

7. Dataset facts

Property	Type I	Type II
Instances	200	200
Sessions per instance	50	50
Serialized tokens, mean	151,705	151,863
Serialized tokens, median	151,505	152,264
Serialized tokens, max	162,693	164,919
Instances over 125,440-token input budget	200 (all)	200 (all)
Mean session gap, old to new evidence	27.1	27.3

- Average of 11.9 turns per session, so about 3K tokens per session.
- The old evidence session index always lies in **[0, 14]** (mean 7.4); the new one always in **[30, 39]** (mean 34.7). This positional regularity comes from the generation pipeline. It is worth noting because a system could in principle exploit position; we will report it and make sure no system sees indices.
- Timestamps are shifted so the final session (query time) falls in 2025.
- Record fields: uid, type, M_old, M_new, explanation, probing_queries{dim1_query, dim2_query, dim3_query}, haystack_session[50][turns{role, content}], timestamps[50], relevant_session_index[2].

8. Evaluation protocol, as implemented officially

8.1 Response generation

SR and PR use the "accurately answer the question" system prompt with `[Question]`. IPA uses the "respond to the user's latest query directly" prompt with `[Latest Query]`. The history is serialized as:

```
=== Session 1 [Time: 2020-04-24 18:15] ===
User: ...
Assistant: ...

=== Session 2 [Time: 2020-05-31 12:43] ===
```

...

8.2 Judge

- One joint call per instance grades all three responses (Appendix B has the full prompt).
- Settings: `temperature=0`, `response_format={"type": "json_object"}`, model Gemini-3.1-flash-lite.
- Output: `{dim1_eval: {reasoning, pass}, dim2_eval: ..., dim3_eval: ...}`.
- Any exception (network or parse) is scored as FAIL on all three dimensions. It is printed but not counted separately.

8.3 Evidence-preserving truncation (B0)

```
input budget = 128,000 - 2,048 (output reserve) - 512 (safety) = 125,440 tokens, applied per probe
phase 1: drop up to 2 sessions from the head (never past S_old)
phase 2: drop up to 2 sessions from the tail (never past S_new)
phase 3: while over budget: drop a random session strictly between S_old and S_new
S_old and S_new are always kept. Token count uses tiktoken (model encoding, o200k_base fallback)
plus chat-format overhead (3 per message + 3 priming).
```

Because the old evidence sits within the first 15 sessions and the new within sessions 30-39, phase 3 always does most of the work. The truncated context therefore keeps the start of the history and loses the middle.

9. The official CUPMem implementation

Package	Key files (lines)	Role
write/	delta_extractor (322), update_resolver (412), invalidation_lanes (503), invalidation_merge (481), invalidation_resolver (185), invalidation_judge (145), stale_linker (191), writer (245), chunker (179)	Session chunking, candidate extraction, same-slot update, latent invalidation across lanes (direct / affected / global), merge and judge
query/	readout (705), support (792), basis_recovery_impl (723), premise_verifier (525), track_hints (429), action_grounding (339), engine (212), retrieval (69)	Query analysis, premise verification, basis recovery, constrained answer
store_layer/	transitions (372), in_memory (232)	Profile store and status transitions
memory/	models (335), schema (186)	Data classes; the Table 8 schema
llm_layer/	client (530), responses_adapter (491)	Chat and Responses API clients with cache and call records
prompt_lib/	templates (467)	All prompts
core/	engine_utils (326), sample_runner (234), temporal (88), config (28)	Orchestration, thresholds

Note on labels: the paper's main text describes a four-way KEEP/STALE/REPLACE/UNKNOWN decision. Appendix F.2 instead describes a local update (ADD/REFINE/REPLACE/NO_OP) followed by a separate latent-invalidation judge. The code, split into update_resolver and invalidation_*, follows the appendix. Our reimplementation will follow the code, and the difference will be documented.

10. Open questions and ambiguities

10.1 Session mode (the central question)

The default `relevant_only` ingests only `haystack_session[i] for i in relevant_session_index`. This gives CUPMem oracle knowledge of where the evidence is, which no baseline has.

10.2 What the paper's cost figure implies (estimate)

GPT-4o-mini costs $0.15 per million input tokens and $0.60 per million output tokens.

- **Paper's figure.** $0.37 per instance corresponds to roughly 2.0-2.5 million tokens processed per instance.
- **Full mode.** CUPMem makes several LLM calls per session (chunking, extraction, update resolution, invalidation lanes, judging). Over 50 sessions of about 3K tokens each, reaching millions of tokens is plausible.
- **relevant_only mode.** Two sessions of about 3K tokens each give roughly 6K tokens of content. Even with 20x call amplification, that is about 120K tokens, or around $0.02.

The cost figure therefore points to full mode, but this is indirect evidence. The plan:

- Run the official code in both modes on the same subset.
- Record tokens and accuracy for each.
- Report relevant_only as an explicit oracle-retrieval ablation.
- Ask the corresponding author.

10.3 Full CUPMem thresholds (cup_mem/core/config.py)

Parameter	Value	Parameter	Value
state_salience_threshold	0.30	candidate_top_k	5
explicit_min_confidence	0.45	latent_invalidation_temperature	0.30
strong_support_min_confidence	0.55	invalidation_lane_max_workers	4
medium_support_min_confidence	0.60	invalidation_merge_max_keep	2
verifier_fallback_threshold	0.45	invalidation_global_recall_top_k	4
query_top_k	8	fallback_top_k	6

10.4 Other open questions

#	Issue	Plan
2	Target temperature unstated; official code sends none (OpenAI default 1.0)	Decide: 0 only, or also default for E1 fidelity
3	Judge errors silently become FAIL	Record, re-judge, report the count (deviation D2)
4	Label sets differ between main text and appendix/code	Follow the code; document
5	Embedding model named only in the code README (all-MiniLM-L6-v2)	Use the same
6	Exact judge API model id string	Confirm when setting up the Gemini key

11. Related work check

Paper	What it does	Overlap with our RQs
StateAuditor (Sun & He, arXiv:2608.01619)	Readout-side audit from stored state to draft response. An LLM proposes old-to-new transitions and deterministic code verifies them. +5.0 points on full STALE.	None on CUPMem reproduction, small adjudicators or multi-hop. Cite it, and consider it as a readout-side baseline.
TEPA (Zhou et al., arXiv:2608.07429)	Keyed precedents; revokes active precedents when fresh evidence contradicts them. 0.950 vs 0.21-0.31 baselines on single-hop settings.	Not evaluated on STALE per its abstract. Its authors state multi-hop exposes bottlenecks beyond their method, which supports RQ5.
The Memory Trust Gap	Mentioned in the blueprint	Not yet located; to do

Part IV: How the code works

12. Design principles

Principle	How it is realised	Why it matters for this paper
Fidelity first	Prompts, serializer, truncation and judge are copied or ported from the official code; tests assert exact strings	Differences in results must come from methods, not from evaluation drift
Every call reproducible	SQLite cache keyed by a hash of model, messages and parameters; seeded randomness	A reproducibility paper must itself be reproducible; reruns are free
One interface for all systems	MemorySystem with reset / ingest / answer	Baselines, official CUPMem, our CUPMem and CUPMem+ are interchangeable
Observability built in	Answer.trace; per-tag usage logs; JSON failures counted	Needed for RQ3 attribution and RQ4 cost
No label leakage	Systems never receive M_old, M_new or evidence indices; only B0 truncation uses indices, as in the paper	Fair comparison
Crash-safe runs	Append-only JSONL outputs; resume skips finished instances	Long API runs fail midway; no lost money
Test without spending	FakeBackend and toy scenarios	Bugs caught before any paid call

13. Module walkthrough

Package layout and data flow:

```
src/stale_repro/
  llm/gateway.py          every LLM call: backends, sqlite cache, JSON decoding, usage accounting
  data/loader.py          raw JSON -> Instance / Session / Probe; deterministic stratified subsets
  memory/base.py          MemorySystem interface; official response prompts
  memory/full_context.py  B0 baseline; official serializer and evidence-preserving truncation
  eval/judge_prompts.py   official judge system prompt (verbatim copy)
  eval/judge.py           joint judge call -> Verdict
  eval/stats.py           cell accuracies, Overall, bootstrap CIs, McNemar, results table
  runner.py               config -> answers.jsonl -> verdicts.jsonl -> summary
scripts/run_experiment.py CLI wrapper

config.yaml -> select_subset -> [parallel per instance]
     system.reset(); system.begin_instance(inst); system.ingest_many(sessions)
     system.answer(dim1), answer(dim2), answer(dim3)          -> runs/<name>/answers.jsonl
  -> Judge.judge(inst, answers)                              -> runs/<name>/verdicts.jsonl
  -> cell_accuracies, bootstrap_ci, results_table            -> runs/<name>/summary.{json,md}
  every LLM call: Gateway.chat -> cache lookup -> backend -> cache store -> usage log
```

13.1 llm/gateway.py: the LLM gateway (242 lines)

Built first because every other component calls it. It hides which provider is used and guarantees caching, JSON handling and usage accounting everywhere.

Element	What it does and why
`Usage`	Dataclass holding prompt_tokens and completion_tokens.
`LLMResponse`	text, model, usage, latency_s, cached flag, parsed JSON (if requested), parse_error. The `ok_json` property is true only when parsing succeeded.
`Backend` (Protocol)	Any object with `complete(model, messages, params) -> (text, Usage)`. This is the seam that lets tests inject a fake.
`OpenAICompatBackend`	Wraps `openai.OpenAI(base_url, api_key, timeout=600, max_retries=5)`. The same class serves OpenAI, vLLM, Gemini's OpenAI endpoint and hosted open-model providers. The SDK's own retries handle rate limits.
`parse_json_loose(text)`	Strips ```json fences, tries json.loads, then falls back to the outermost {...} or [...] span. Raises ValueError if nothing parses. Small models often wrap JSON in prose, so strict parsing would misattribute failures.

Element	What it does and why
`_Cache`	SQLite table (key TEXT PRIMARY KEY, value TEXT). `check_same_thread=False` plus a lock makes it safe for the runner's thread pool. Values store text, usage and latency.
`UsageTotals`	Per-tag counters: calls, cached_calls, tokens, latency, json_failures.
`Gateway.chat(...)`	Main entry point. Accepts a string or message list, an optional system prompt, `json_mode` or `json_schema`, a model override, and decoding params (default temperature 0). It builds `response_format`, calls `_call`, then parses JSON if requested. On failure it retries once with a repair turn ("Your previous reply was not valid JSON..."), tagged `.repair`. If that also fails, `json_failures` is incremented.
`cache_key(...)`	SHA-256 of canonical JSON (sorted keys) of {cache version, model, messages, params}. Any change in prompt, model or decoding parameters gives a new key. Bumping CACHE_VERSION invalidates everything.
`_call(...)`	Cache lookup; on a miss, time the backend call and store the result. Always records usage, including cache hits flagged as cached, so cost reports can separate real spend from replay.
`_record / _bump`	Update per-tag totals and append one JSONL line per call: ts, tag, model, key prefix, cached, tokens, latency.
`summary() / reset_totals()`	Snapshot or clear the per-tag totals; the runner stores them in summary.json.
`gateway_from_config(cfg)`	Builds a Gateway from a YAML block: model, base_url, api_key_env (name of the env variable, never the key itself), params.

```
gw = gateway_from_config({"model": "gpt-4o-mini", "api_key_env": "OPENAI_API_KEY"}, "cache", "runs/x/calls.jsonl
    ")
r = gw.chat([{"role": "user", "content": prompt}], system=SYS, json_mode=True, tag="judge", temperature=0)
if r.ok_json: use(r.parsed)
print(gw.summary())   # {"judge": {"calls": 1, "cached_calls": 0, "prompt_tokens": ..., "json_failures": 0}}
```

13.2 data/loader.py: data model and subsets (109 lines)

Element	What it does and why
`DIMS, DIM_NAMES, TYPE_NAMES`	Constants mapping dim1/2/3 to SR/PR/IPA and T1/T2 to Type I/II.
`Session`	index, timestamp, turns. `render()` produces User:/Assistant: lines.
`Probe`	uid, dim, text, query_time (the last session's timestamp). `is_task` is true for dim3, which selects the IPA prompt.
`Instance`	uid, type, m_old, m_new, explanation, probes (dict), sessions, idx_old, idx_new, meta (any unknown fields, kept rather than dropped). Helpers: evidence_indices, raw_sessions(), timestamps().
`parse_record(rec)`	Converts one raw JSON record into an Instance.
`load_stale(path)`	Loads the file. Accepts a JSON list or an object with a 'data' list, like the official scripts.
`select_subset(...)`	Deterministic stratified subset. Within each type, instances are sorted by `sha256(f'{seed}:{uid}')` and the first n are taken. The selection therefore depends only on (seed, n) and not on file order, and larger subsets contain smaller ones. An explicit uid list is also supported.

13.3 memory/base.py: the common interface (58 lines)

- `SYSTEM_QA` / `SYSTEM_TASK`: the two official response-generation system prompts, verbatim.
- `build_answer_prompt(history, probe)`: returns (system, user) exactly as the official code does, using `[Question]` for SR/PR and `[Latest Query]` for IPA. Memory systems that produce their own context can still reuse this so the probe framing stays identical.
- `Answer(text, trace)`: trace is a free-form dict. CUPMem will put store snapshots, candidate sets and adjudication labels here for failure attribution.
- `MemorySystem`: `reset()`, `begin_instance(inst)`, abstract `ingest(session)` and `answer(probe)`, plus `ingest_many`. The docstring on begin_instance makes the rule explicit: systems must not read labels.

Only B0 uses it, to get evidence indices for truncation, exactly as the paper does.

13.4 memory/full_context.py: B0 baseline (133 lines)

Element	What it does and why
`format_haystack(sessions, timestamps)`	Character-for-character port of the official serializer. A test compares the exact output string.
`_encoder(model)`	Cached tiktoken encoder: the model's own encoding if tiktoken knows it, else o200k_base, as in the official code.
`count_message_tokens`	Official chat-format estimate: 3 tokens per message plus content tokens plus 3 for reply priming.
`trim_evidence_preserving(...)`	The three-phase algorithm in Section 8.3, operating on copies and re-counting tokens after each drop, as in the official code. It returns the trimmed sessions and timestamps, the final token count and the number removed. Phase 3 draws from an injected `random.Random`, which makes it deterministic.
`FullContext.__init__`	gateway, context_limit (None disables truncation), reserved_output=2048, safety_margin=512, seed, optional max_tokens.
`FullContext.ingest`	Appends the session; B0 has no memory processing.
`FullContext.answer`	If a limit is set, derives a per-(seed, uid, dim) RNG seed from SHA-256 and truncates. Then builds the official prompt and calls the gateway with tag `B0_full_context.answer`. The trace records sessions removed, estimated request tokens, usage and cache status.

13.5 eval/judge_prompts.py and eval/judge.py: the judge (136 lines)

- **Prompt provenance.** judge_prompts.py was produced by a script that reads the official file and writes it back with a header naming the source commit and licence. Retyping a 40-line prompt risks invisible differences, such as a changed space or quote, that would break comparability.
- `build_judge_user_prompt(inst, responses)`: the official f-string, reproduced exactly, including the quotes around M_old and M_new and the separator lines.
- `Verdict`: uid, type, passed {dim: bool}, reasoning {dim: str}, error. Includes to_dict/from_dict for JSONL.
- `_as_bool`: accepts true booleans and the strings "true"/"false". Anything else counts as false.
- `parse_judge_output`: requires all three dimK_eval blocks with a pass field. A missing key raises an error instead of guessing.
- `Judge.judge(inst, responses)`: one call with the official system prompt, json_mode and temperature 0, tagged `judge`. Any failure yields a Verdict with all False and a non-empty `error`. The runner re-judges errored verdicts on the next run and reports the count.

13.6 eval/stats.py: scoring and statistics (98 lines)

Function	What it does and why
`cell_accuracies(verdicts)`	Accuracy for each of the six (type, dim) cells, plus `overall` = unweighted mean of the cells, matching the paper. This is not the mean over all 1,200 queries (identical only when types are balanced).
`bootstrap_ci(...)`	2,000 percentile-bootstrap resamples (default) of **instances within each type**. The three probes of one instance share the same memory state, so they are correlated. Resampling individual queries would make confidence intervals too narrow. Stratifying by type keeps the design balanced. The overall CI is computed on the resampled six-cell mean.
`mcnemar_exact(a, b)`	Exact two-sided McNemar test on paired binary outcomes: counts discordant pairs and uses the binomial distribution. It is exact, so there is no chi-square approximation issue with small counts.
`paired_outcomes(va, vb, dim, type_)`	Aligns two systems' verdicts on common uids, optionally restricted to one dimension or type, ready for McNemar.
`results_table(verdicts)`	Markdown row in the paper's column order, with [lo, hi] CIs and a judge-error count.

13.7 runner.py and scripts/run_experiment.py: running experiments (161 lines)

Element	What it does and why
`_registry()`	Maps a config `kind` to a MemorySystem class. Currently only `full_context`; new systems register here.
`run_system_on_instance(system, inst, mode)`	reset, then begin_instance, then ingest all 50 sessions (or only the two evidence sessions in `relevant_only` mode, mirroring the official CUPMem option), then answer dim1-3. Returns uid, type, answers and traces.
`run(cfg)`	(1) Create runs/<name> and save the config. (2) Load and subset data. (3) Answer: skip uids already in answers.jsonl, then run the rest in a thread pool, with a fresh system per task sharing one gateway. (4) Judge: skip uids with an error-free verdict. (5) Take the last verdict per uid, compute accuracies and CIs, and write summary.json (including per-tag usage for system and judge) and summary.md.
`_JsonlWriter`	Lock-protected append. Each line is complete, so a crash loses at most in-flight instances.
`_pool(fn, items, workers)`	ThreadPoolExecutor with a tqdm bar. It collects all exceptions, prints up to 5, and raises one summary error telling you to rerun to resume. Threads (rather than asyncio) fit because the work is network-bound and the memory-system code stays simple and synchronous.
`scripts/run_experiment.py`	Loads .env, reads the YAML, and applies `--n-per-type` (appended to the run name so smoke tests never mix with real runs), `--workers`, `--out-root` and `--cache-dir`. Prints the headline numbers.

Outputs of one run, in `runs//`: config.yaml, answers.jsonl (answers and per-probe traces), verdicts.jsonl, summary.json, summary.md, llm_calls.jsonl and judge_calls.jsonl.

Part V: Testing, deviations, reproduction, next steps

14. Testing

Strategy. `FakeBackend` implements the Backend protocol by calling a scripted Python function and records every request (model, messages, params). Tests can therefore assert exactly what would be sent to a real API, at zero cost. A `make_gateway` fixture builds a Gateway with a temporary cache and log.

Toy scenarios mirror the paper's conflict types and are built by `toy_record` as six-session instances with filler sessions:

- **toy-seattle (T1)**: "lived in Seattle for years" (session 1), then "set up electricity and internet at my new apartment in Portland" (session 4).
- **toy-leg (T2)**: "cycle to work every day, 8 km" (session 1), then "leg in a cast for six weeks after the fracture" (session 4).

Test	Verifies
test_cache_hit_skips_backend	Second identical call is served from cache; backend called once; totals count 2 calls, 1 cached
test_cache_key_depends_on_params	Different temperature or model triggers new backend calls
test_cache_persists_across_gateways	A new Gateway on the same file reuses stored responses (the replacement backend is never called)
test_json_mode_parses_fenced_output	```json fences parsed; response_format set to json_object
test_json_repair_retry_then_failure_is_counted	Two bad replies: one retry, json_failures incremented
test_json_repair_retry_succeeds	Bad reply then good reply: parsed result returned
test_json_schema_sets_response_format	json_schema produces a strict json_schema response_format
test_usage_log_written	JSONL call log contains the caller's tag
test_parse_json_loose [3 cases]	Plain JSON; JSON inside prose; fenced array
test_parse_json_loose_raises	Text without JSON raises ValueError
test_loader_roundtrip	Toy file loads; types, evidence indices, content and is_task are correct
test_subset_is_deterministic_and_stratified	Same subset regardless of input order; 3 per type
test_format_haystack_matches_official	Exact serialized string equals the official format
test_truncation_preserves_evidence	20 long sessions trimmed to 4,000 tokens: both evidence sessions kept, sessions removed, budget respected
test_b0_sends_official_prompts	Correct system prompt per probe; [Latest Query] for IPA; evidence present in context
test_relevant_only_mode_ingests_two_sessions	Oracle mode serializes exactly 2 sessions
test_judge_prompt_and_parse	Official system prompt sent byte-for-byte; temperature 0; user prompt format; verdict parsed
test_judge_malformed_output_is_error_not_silent	Incomplete judge JSON produces an error field and all-False
test_overall_is_mean_of_six_cells	Hand-computed cell and overall values; degenerate CI for a constant cell
test_mcnemar	Identical outcomes give p=1; 10 vs 0 discordant gives p<0.01
test_runner_end_to_end	Full config run with fake system and judge; overall 1.0; rerun resumes with identical result; output files exist

```
> .venv/Scripts/python -m pytest -q
.......................                                  [100%]
23 passed in 14.73s
```

15. Deviations from the official code

Every deviation is intentional, minimal, and logged in notes/reproducibility_log.md.

ID	Where	Official	Ours	Reason
D1	B0 truncation phase 3	Unseeded random.choice	RNG seeded by SHA-256(seed, uid, dim)	Exact reruns
D2	Judge errors	Silently FAIL	error field; re-judged on resume; count reported	Transparency
D3	Target temperature	Not set (API default 1.0)	0 in configs, configurable	Determinism and caching; E1 choice pending
D4	Truncation token counting	tiktoken, o200k fallback	Same	none

16. How to reproduce everything

```
# 1. environment
python -m venv .venv
.venv/Scripts/python -m pip install -e ".[dev]"            # use .venv/bin/python on Linux/macOS
cp .env.example .env                                         # OPENAI_API_KEY, GEMINI_API_KEY

# 2. data and official code
curl -L -o data/stale/T1_T2_400_FULL.json \
  https://huggingface.co/datasets/STALEproj/STALE/resolve/main/T1_T2_400_FULL.json
git clone https://github.com/icedreamc/STALE external/STALE

# 3. offline tests
.venv/Scripts/python -m pytest

# 4. smoke test (2 per type), then 50-instance dev run
.venv/Scripts/python scripts/run_experiment.py configs/b0_gpt4omini_dev.yaml --n-per-type 2
.venv/Scripts/python scripts/run_experiment.py configs/b0_gpt4omini_dev.yaml

# 5. rebuild this report
.venv/Scripts/python notes/make_week1_report.py
```

17. Open decisions, next steps, budget

17.1 Decisions needed

- API keys: OpenAI (GPT-4o-mini target) and Gemini (judge); confirm the judge model id.
- Target temperature for E1: 0 only, or also the official default?
- Initialise git and make the first commit.
- Contact the corresponding author about the session mode. The Gmail connector is not yet authorised; a draft can be written instead.

17.2 Next steps

#	Task	Output	Blueprint
1	Smoke test, then B0 on the dev subset	First real numbers vs paper's GPT-4o-mini row	E1 / week 3
2	Wrap official CUPMem as a MemorySystem; run in full and relevant_only modes	Answer to the central question; per-mode cost	RQ1
3	Reconcile label sets against cup_mem/write/	Spec for our reimplementation	Weeks 4-5
4	Independent CUPMem: store, extract, search, adjudicate, readout	System M	Weeks 4-5
5	B1 naive RAG, B2 Mem0	Baselines	Week 3
6	Attribution from traces	Key failure-analysis figure	RQ3
7	Open-model backbones 3B-32B	Accuracy vs size	E2 / weeks 7-8

17.3 Budget estimates so far

Run	Estimate	Basis
B0 GPT-4o-mini, dev subset (50)	~$3	~125K input tokens x 3 probes x 50 x $0.15/M
B0 GPT-4o-mini, full (400)	~$23	Same, 400 instances
Official CUPMem full mode, full (400)	~$150	Paper's $0.37/instance; to be measured
Judge	small	One call per instance with short inputs

Appendix A: Complete source code

Every file below is included in full and read from disk when this PDF is built, so it always matches the code. Lines longer than 112 characters are wrapped with a 4-space continuation indent.

A.1 pyproject.toml (27 lines): Packaging and dependencies

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "stale-repro"
version = "0.1.0"
description = "Reproducing and stress-testing STALE / CUPMem (write-side state adjudication for agent memory)"
requires-python = ">=3.10"
dependencies = [
    "openai>=1.40",
    "tiktoken>=0.7",
    "numpy>=1.26",
    "pyyaml>=6.0",
    "python-dotenv>=1.0",
    "tqdm>=4.66",
]

[project.optional-dependencies]
dev = ["pytest>=8.0"]
embed = ["sentence-transformers>=3.0"]

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

A.2 configs/b0_gpt4omini_dev.yaml (22 lines): B0 development experiment

```
# E1 / B0 development run: plain long-context GPT-4o-mini on a 50-instance subset.
# Paper reference (Table 2, full 400): T1 30.0/0.0/11.0, T2 9.5/0.0/1.5, Overall 8.7
name: b0_gpt4omini_dev
data: data/stale/T1_T2_400_FULL.json
subset: {n_per_type: 25, seed: 0}
session_mode: full
workers: 8

system:
  kind: full_context
  llm:
    model: gpt-4o-mini
    api_key_env: OPENAI_API_KEY
    params: {temperature: 0}   # paper does not state target temperature; official code uses API default
  args:
    context_limit: 128000      # official TOKEN_LIMIT; triggers evidence-preserving truncation

judge:
  llm:
    model: gemini-3.1-flash-lite
    base_url: https://generativelanguage.googleapis.com/v1beta/openai/
    api_key_env: GEMINI_API_KEY
```

A.3 src/stale_repro/llm/gateway.py (242 lines): LLM gateway

```
"""LLM gateway: one OpenAI-compatible entry point for every model call.

Responsibilities
- Disk cache keyed by sha256(model, messages, decoding params). Reruns are free
  and bit-for-bit reproducible, and the cache itself can be released.
- Structured output: ``json_mode`` (json_object) or ``json_schema`` (strict
  schema; supported by OpenAI and by vLLM's OpenAI server). Parse failures are
  recorded, never silently swallowed, because they are a finding for small models.
- Usage accounting: every call (cached or not) is appended to a JSONL log with a
  caller ``tag`` such as ``cupmem.extract``, which feeds the cost analysis (E4).
"""
from __future__ import annotations

import hashlib
import json
import os
import re
import sqlite3
import threading
import time
from dataclasses import asdict, dataclass, field
from pathlib import Path
from typing import Any, Protocol

CACHE_VERSION = 1


@dataclass
class Usage:
    prompt_tokens: int = 0
    completion_tokens: int = 0


@dataclass
class LLMResponse:
    text: str
    model: str
    usage: Usage
    latency_s: float
    cached: bool = False
    parsed: Any = None
    parse_error: str | None = None

    @property
    def ok_json(self) -> bool:
        return self.parse_error is None and self.parsed is not None


class Backend(Protocol):
    def complete(self, model: str, messages: list[dict], params: dict) -> tuple[str, Usage]: ...


class OpenAICompatBackend:
    """Works with api.openai.com, vLLM, and any OpenAI-compatible host."""

    def __init__(self, base_url: str | None = None, api_key: str | None = None,
                 timeout: float = 600.0, max_retries: int = 5):
        from openai import OpenAI

        self.client = OpenAI(base_url=base_url, api_key=api_key or "EMPTY",
                             timeout=timeout, max_retries=max_retries)

    def complete(self, model: str, messages: list[dict], params: dict) -> tuple[str, Usage]:
        resp = self.client.chat.completions.create(model=model, messages=messages, **params)
        text = resp.choices[0].message.content or ""
        u = resp.usage
        usage = Usage(u.prompt_tokens, u.completion_tokens) if u else Usage()
        return text, usage


_FENCE = re.compile(r"```(?:json)?\s*(.*?)\s*```", re.S)


def parse_json_loose(text: str) -> Any:
    """Parse JSON, tolerating code fences and leading/trailing prose."""
    s = text.strip()
    m = _FENCE.search(s)
    if m:
        s = m.group(1)
    try:
        return json.loads(s)
    except json.JSONDecodeError:
        pass
    # fall back to the outermost {...} or [...] span
    for open_c, close_c in (("{", "}"), ("[", "]")):
        i, j = s.find(open_c), s.rfind(close_c)
```

```
        if 0 <= i < j:
            try:
                return json.loads(s[i:j + 1])
            except json.JSONDecodeError:
                continue
    raise ValueError("no parseable JSON found")


class _Cache:
    def __init__(self, path: Path):
        path.parent.mkdir(parents=True, exist_ok=True)
        self._conn = sqlite3.connect(str(path), check_same_thread=False)
        self._conn.execute("CREATE TABLE IF NOT EXISTS cache (key TEXT PRIMARY KEY, value TEXT)")
        self._conn.commit()
        self._lock = threading.Lock()

    def get(self, key: str) -> dict | None:
        with self._lock:
            row = self._conn.execute("SELECT value FROM cache WHERE key=?", (key,)).fetchone()
        return json.loads(row[0]) if row else None

    def put(self, key: str, value: dict) -> None:
        with self._lock:
            self._conn.execute("INSERT OR REPLACE INTO cache VALUES (?, ?)", (key, json.dumps(value)))
            self._conn.commit()


@dataclass
class UsageTotals:
    calls: int = 0
    cached_calls: int = 0
    prompt_tokens: int = 0
    completion_tokens: int = 0
    latency_s: float = 0.0
    json_failures: int = 0


class Gateway:
    def __init__(self, backend: Backend, model: str, cache_path: str | Path | None = None,
                 log_path: str | Path | None = None, default_params: dict | None = None,
                 json_repair_retries: int = 1):
        self.backend = backend
        self.model = model
        self.cache = _Cache(Path(cache_path)) if cache_path else None
        self.log_path = Path(log_path) if log_path else None
        if self.log_path:
            self.log_path.parent.mkdir(parents=True, exist_ok=True)
        self.default_params = {"temperature": 0.0, **(default_params or {})}
        self.json_repair_retries = json_repair_retries
        self.totals: dict[str, UsageTotals] = {}
        self._lock = threading.Lock()

    # ------------------------------------------------------------------ public
    def chat(self, messages: list[dict] | str, *, tag: str = "untagged", system: str | None = None,
             json_mode: bool = False, json_schema: dict | None = None,
             model: str | None = None, **params: Any) -> LLMResponse:
        if isinstance(messages, str):
            messages = [{"role": "user", "content": messages}]
        if system:
            messages = [{"role": "system", "content": system}, *messages]
        model = model or self.model
        call_params = {**self.default_params, **params}
        if json_schema is not None:
            call_params["response_format"] = {
                "type": "json_schema",
                "json_schema": {"name": json_schema.get("title", "output"), "schema": json_schema, "strict": True},
            }
        elif json_mode:
            call_params["response_format"] = {"type": "json_object"}

        resp = self._call(model, messages, call_params, tag)
        if json_mode or json_schema is not None:
            self._attach_json(resp)
            retries = self.json_repair_retries
            while not resp.ok_json and retries > 0:
                retries -= 1
                repair = [*messages, {"role": "assistant", "content": resp.text},
                          {"role": "user", "content": "Your previous reply was not valid JSON. "
                                                      "Reply again with only the JSON object."}]
                resp = self._call(model, repair, call_params, tag + ".repair")
                self._attach_json(resp)
            if not resp.ok_json:
                self._bump(tag, json_failure=True)
        return resp

    def summary(self) -> dict[str, dict]:
        with self._lock:
            return {k: asdict(v) for k, v in self.totals.items()}

    def reset_totals(self) -> None:
```

```
        with self._lock:
            self.totals = {}

    # ------------------------------------------------------------------ private
    @staticmethod
    def cache_key(model: str, messages: list[dict], params: dict) -> str:
        blob = json.dumps({"v": CACHE_VERSION, "model": model, "messages": messages, "params": params},
                          sort_keys=True, ensure_ascii=False)
        return hashlib.sha256(blob.encode("utf-8")).hexdigest()

    def _call(self, model: str, messages: list[dict], params: dict, tag: str) -> LLMResponse:
        key = self.cache_key(model, messages, params)
        hit = self.cache.get(key) if self.cache else None
        if hit is not None:
            resp = LLMResponse(text=hit["text"], model=model, usage=Usage(**hit["usage"]),
                               latency_s=hit["latency_s"], cached=True)
        else:
            t0 = time.perf_counter()
            text, usage = self.backend.complete(model, messages, params)
            resp = LLMResponse(text=text, model=model, usage=usage, latency_s=time.perf_counter() - t0)
            if self.cache:
                self.cache.put(key, {"text": text, "usage": asdict(usage), "latency_s": resp.latency_s})
        self._record(tag, key, resp)
        return resp

    @staticmethod
    def _attach_json(resp: LLMResponse) -> None:
        try:
            resp.parsed, resp.parse_error = parse_json_loose(resp.text), None
        except ValueError as e:
            resp.parsed, resp.parse_error = None, str(e)

    def _bump(self, tag: str, resp: LLMResponse | None = None, json_failure: bool = False) -> None:
        with self._lock:
            t = self.totals.setdefault(tag, UsageTotals())
            if json_failure:
                t.json_failures += 1
                return
            assert resp is not None
            t.calls += 1
            t.cached_calls += int(resp.cached)
            t.prompt_tokens += resp.usage.prompt_tokens
            t.completion_tokens += resp.usage.completion_tokens
            t.latency_s += resp.latency_s

    def _record(self, tag: str, key: str, resp: LLMResponse) -> None:
        self._bump(tag, resp)
        if self.log_path:
            line = json.dumps({"ts": time.time(), "tag": tag, "model": resp.model, "key": key[:16],
                               "cached": resp.cached, "prompt_tokens": resp.usage.prompt_tokens,
                               "completion_tokens": resp.usage.completion_tokens,
                               "latency_s": round(resp.latency_s, 3)})
            with self._lock, self.log_path.open("a", encoding="utf-8") as f:
                f.write(line + "\n")


def gateway_from_config(cfg: dict, cache_dir: str | Path = "cache", log_path: str | Path | None = None) -> Gateway:
    """Build a Gateway from a config block such as

        {"model": "gpt-4o-mini", "base_url": null, "api_key_env": "OPENAI_API_KEY",
         "params": {"temperature": 0}}
    """
    api_key = os.environ.get(cfg.get("api_key_env", "OPENAI_API_KEY"), "")
    backend = OpenAICompatBackend(base_url=cfg.get("base_url"), api_key=api_key)
    return Gateway(backend, model=cfg["model"], cache_path=Path(cache_dir) / "llm_cache.sqlite",
                   log_path=log_path, default_params=cfg.get("params"))
```

A.4 src/stale_repro/data/loader.py (109 lines): Data model and subsets

```
"""STALE data loading.

Record format (HF: STALEproj/STALE, file T1_T2_400_FULL.json) -- a JSON list of:
    uid, type ("T1"|"T2"), M_old, M_new, explanation,
    probing_queries {dim1_query, dim2_query, dim3_query},
    haystack_session [50 sessions, each a list of {role, content}],
    timestamps [50 strings "YYYY-MM-DD HH:MM"],
    relevant_session_index [idx_old, idx_new]
"""
from __future__ import annotations

import hashlib
import json
from dataclasses import dataclass, field
from pathlib import Path
from typing import Iterable

DIMS = ("dim1", "dim2", "dim3")
DIM_NAMES = {"dim1": "SR", "dim2": "PR", "dim3": "IPA"}
TYPE_NAMES = {"T1": "Type I", "T2": "Type II"}


@dataclass
class Session:
    index: int
    timestamp: str
    turns: list[dict]  # [{"role": "user"|"assistant", "content": str}]

    def render(self) -> str:
        return "\n".join(f"{'User' if t['role'] == 'user' else 'Assistant'}: {t['content']}" for t in self.turns)


@dataclass
class Probe:
    uid: str
    dim: str   # dim1 | dim2 | dim3
    text: str
    query_time: str = ""  # timestamp of the last session (the paper's query_time)

    @property
    def is_task(self) -> bool:
        """dim3 (IPA) is posed as the user's latest request, not a question about the user."""
        return self.dim == "dim3"


@dataclass
class Instance:
    uid: str
    type: str
    m_old: str
    m_new: str
    explanation: str
    probes: dict[str, Probe]
    sessions: list[Session]
    idx_old: int
    idx_new: int
    meta: dict = field(default_factory=dict)

    @property
    def evidence_indices(self) -> tuple[int, int]:
        return self.idx_old, self.idx_new

    def raw_sessions(self) -> list[list[dict]]:
        return [s.turns for s in self.sessions]

    def timestamps(self) -> list[str]:
        return [s.timestamp for s in self.sessions]


def parse_record(rec: dict) -> Instance:
    ts = rec.get("timestamps", [])
    sessions = [Session(i, ts[i] if i < len(ts) else "", turns)
                for i, turns in enumerate(rec["haystack_session"])]
    query_time = ts[-1] if ts else ""
    probes = {d: Probe(rec["uid"], d, rec["probing_queries"][f"{d}_query"], query_time) for d in DIMS}
    idx_old, idx_new = rec["relevant_session_index"]
    known = {"uid", "type", "M_old", "M_new", "explanation", "probing_queries",
             "haystack_session", "timestamps", "relevant_session_index"}
    return Instance(uid=rec["uid"], type=rec["type"], m_old=rec["M_old"], m_new=rec["M_new"],
                    explanation=rec["explanation"], probes=probes, sessions=sessions,
                    idx_old=int(idx_old), idx_new=int(idx_new),
                    meta={k: v for k, v in rec.items() if k not in known})
```

```
def load_stale(path: str | Path) -> list[Instance]:
    with Path(path).open(encoding="utf-8") as f:
        payload = json.load(f)
    records = payload["data"] if isinstance(payload, dict) else payload
    return [parse_record(r) for r in records]


def _stable_rank(uid: str, seed: int) -> str:
    return hashlib.sha256(f"{seed}:{uid}".encode()).hexdigest()


def select_subset(instances: Iterable[Instance], n_per_type: int | None = None,
                  types: Iterable[str] = ("T1", "T2"), seed: int = 0,
                  uids: Iterable[str] | None = None) -> list[Instance]:
    """Deterministic stratified subset. Independent of file order, so the
    same (n_per_type, seed) always selects the same instances."""
    instances = list(instances)
    if uids is not None:
        wanted = set(uids)
        return [x for x in instances if x.uid in wanted]
    out: list[Instance] = []
    for t in types:
        pool = sorted((x for x in instances if x.type == t), key=lambda x: _stable_rank(x.uid, seed))
        out.extend(pool if n_per_type is None else pool[:n_per_type])
    return out
```

A.5 src/stale_repro/memory/base.py (58 lines): MemorySystem interface

```
"""Common interface for every memory system under test (B0..B3, M, M+).

Lifecycle per STALE instance:
    system.reset()
    for session in instance.sessions: system.ingest(session)
    for probe in instance.probes.values(): system.answer(probe)

``Answer.trace`` carries whatever intermediate state a system wants to expose;
CUPMem uses it for stage-wise failure attribution.
"""
from __future__ import annotations

from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import Any

from ..data.loader import Instance, Probe, Session

# Response-generation prompts, verbatim from the paper (App. E.1) and the
# official STALE/Evaluation/run_target_model.py.
SYSTEM_QA = ("You are a helpful assistant. Review the following conversation history "
             "with the user, then accurately answer the question.")
SYSTEM_TASK = ("You are a helpful assistant. Review the following conversation history "
               "with the user, then respond to the user's latest query directly.")


def build_answer_prompt(history_text: str, probe: Probe) -> tuple[str, str]:
    if probe.is_task:
        return SYSTEM_TASK, f"[Conversation History]\n{history_text}\n\n[Latest Query]\n{probe.text}"
    return SYSTEM_QA, f"[Conversation History]\n{history_text}\n\n[Question]\n{probe.text}"


@dataclass
class Answer:
    text: str
    trace: dict[str, Any] = field(default_factory=dict)


class MemorySystem(ABC):
    name: str = "base"

    def reset(self) -> None:  # noqa: B027 - optional hook
        """Clear all state before a new instance."""

    def begin_instance(self, instance: Instance) -> None:
        """Optional hook. Systems must NOT read labels (M_old/M_new/evidence
        indices) from here; the runner only calls it so B0 can apply the
        paper's evidence-preserving truncation, which the paper also grants."""

    @abstractmethod
    def ingest(self, session: Session) -> None: ...

    @abstractmethod
    def answer(self, probe: Probe) -> Answer: ...

    def ingest_many(self, sessions: list[Session]) -> None:
        for s in sessions:
            self.ingest(s)
```

A.6 src/stale_repro/memory/full_context.py (133 lines): B0 baseline

```
"""B0: plain long-context model.

Mirrors the official STALE/Evaluation/run_target_model.py:
  - identical history serialization (``format_haystack``) and prompts;
  - identical 3-phase evidence-preserving truncation (drop <=2 head sessions,
    then <=2 tail sessions, then random sessions strictly between S_old and
    S_new) applied per probe, since query lengths differ.
Deviation (documented in notes/reproducibility_log.md): the official phase-3
uses an unseeded ``random.choice``; we use a seeded RNG per (uid, dim).
"""
from __future__ import annotations

import hashlib
import random
from functools import lru_cache

from ..data.loader import Instance, Probe, Session
from ..llm.gateway import Gateway
from .base import Answer, MemorySystem, build_answer_prompt


def format_haystack(sessions: list[list[dict]], timestamps: list[str] | None = None) -> str:
    out = ""
    for idx, session in enumerate(sessions):
        if not session:
            continue
        time_str = f" [Time: {timestamps[idx]}]" if timestamps and idx < len(timestamps) else ""
        out += f"\n=== Session {idx + 1}{time_str} ===\n"
        for turn in session:
            role = "User" if turn["role"] == "user" else "Assistant"
            out += f"{role}: {turn['content']}\n"
    return out


@lru_cache(maxsize=8)
def _encoder(model: str):
    import tiktoken
    try:
        return tiktoken.encoding_for_model(model)
    except KeyError:
        return tiktoken.get_encoding("o200k_base")


def count_message_tokens(system: str, user: str, model: str) -> int:
    enc = _encoder(model)
    # chat-format overhead as in the official num_tokens_from_messages
    return 3 + len(enc.encode(system)) + 3 + len(enc.encode(user)) + 3


def trim_evidence_preserving(sessions: list[list[dict]], timestamps: list[str], idx_old: int, idx_new: int,
                             probe: Probe, model: str, token_limit: int, rng: random.Random
                             ) -> tuple[list[list[dict]], list[str], int, int]:
    """Returns (sessions, timestamps, request_tokens, removed_count)."""
    sessions, timestamps = list(sessions), list(timestamps)

    def tokens() -> int:
        sys_p, user_p = build_answer_prompt(format_haystack(sessions, timestamps), probe)
        return count_message_tokens(sys_p, user_p, model)

    cur, removed = tokens(), 0
    if cur <= token_limit:
        return sessions, timestamps, cur, 0

    for _ in range(min(2, idx_old)):                      # phase 1: head
        if cur <= token_limit:
            break
        sessions.pop(0); timestamps.pop(0) if timestamps else None
        removed += 1; idx_old -= 1; idx_new -= 1
        cur = tokens()
    if cur <= token_limit:
        return sessions, timestamps, cur, removed

    for _ in range(min(2, len(sessions) - 1 - idx_new)):   # phase 2: tail
        if cur <= token_limit:
            break
        sessions.pop(); timestamps.pop() if timestamps else None
        removed += 1
        cur = tokens()
    if cur <= token_limit:
        return sessions, timestamps, cur, removed

    middle = list(range(idx_old + 1, idx_new))             # phase 3: between evidence
    while cur > token_limit and middle:
        drop = rng.choice(middle)
```

```
            sessions.pop(drop)
            if timestamps and drop < len(timestamps):
                timestamps.pop(drop)
            removed += 1; idx_new -= 1
            middle = list(range(idx_old + 1, idx_new))
            cur = tokens()
        return sessions, timestamps, cur, removed


    class FullContext(MemorySystem):
        name = "B0_full_context"

        def __init__(self, gateway: Gateway, context_limit: int | None = None,
                     reserved_output: int = 2048, safety_margin: int = 512, seed: int = 0,
                     max_tokens: int | None = None):
            self.gw = gateway
            self.input_limit = (context_limit - reserved_output - safety_margin) if context_limit else None
            self.seed = seed
            self.max_tokens = max_tokens
            self.reset()

        def reset(self) -> None:
            self.sessions: list[Session] = []
            self.evidence: tuple[int, int] | None = None
            self.uid = ""

        def begin_instance(self, instance: Instance) -> None:
            self.uid = instance.uid
            self.evidence = instance.evidence_indices  # used only for truncation, as in the paper

        def ingest(self, session: Session) -> None:
            self.sessions.append(session)

        def answer(self, probe: Probe) -> Answer:
            raw = [s.turns for s in self.sessions]
            ts = [s.timestamp for s in self.sessions]
            removed, req_tokens = 0, None
            if self.input_limit is not None:
                if self.evidence is None:
                    raise RuntimeError("truncation needs evidence indices; call begin_instance()")
                h = int(hashlib.sha256(f"{self.seed}:{probe.uid}:{probe.dim}".encode()).hexdigest()[:8], 16)
                raw, ts, req_tokens, removed = trim_evidence_preserving(
                    raw, ts, *self.evidence, probe, self.gw.model, self.input_limit, random.Random(h))
            system, user = build_answer_prompt(format_haystack(raw, ts), probe)
            params = {"max_tokens": self.max_tokens} if self.max_tokens else {}
            resp = self.gw.chat([{"role": "user", "content": user}], system=system,
                                tag=f"{self.name}.answer", **params)
            return Answer(resp.text, trace={"removed_sessions": removed, "request_tokens_est": req_tokens,
                                            "usage": vars(resp.usage), "cached": resp.cached})
```

A.7 src/stale_repro/eval/judge.py (90 lines): Judge

```
"""LLM-as-judge, identical in prompt and decoding to the official evaluator
(STALE/Evaluation/full_eval_performance.py): one joint call per instance that
grades all three probe responses, temperature 0, json_object output.

Deviation: the official code silently scores judge errors as FAIL. We keep
``error`` on the verdict so errors can be counted and re-run instead.
"""
from __future__ import annotations

from dataclasses import dataclass, field

from ..data.loader import DIMS, Instance
from ..llm.gateway import Gateway
from .judge_prompts import SYSTEM_PROMPT_ALL_IN_ONE_JUDGE


def build_judge_user_prompt(inst: Instance, responses: dict[str, str]) -> str:
    q = {d: inst.probes[d].text for d in DIMS}
    return f"""
[Ground Truth Context]
- M_old: "{inst.m_old}"
- M_new: "{inst.m_new}"
- Hidden Logic: {inst.explanation}

------------------------------------------------
[Dimension 1: Explicit Probing]
Question 1: {q['dim1']}
Target Model Response 1: {responses['dim1']}

------------------------------------------------
[Dimension 2: Adversarial Robustness]
Question 2: {q['dim2']}
Target Model Response 2: {responses['dim2']}

------------------------------------------------
[Dimension 3: Implicit Task]
Question 3: {q['dim3']}
Target Model Response 3: {responses['dim3']}
"""


@dataclass
class Verdict:
    uid: str
    type: str
    passed: dict[str, bool]
    reasoning: dict[str, str] = field(default_factory=dict)
    error: str | None = None

    def to_dict(self) -> dict:
        return {"uid": self.uid, "type": self.type, "passed": self.passed,
                "reasoning": self.reasoning, "error": self.error}

    @classmethod
    def from_dict(cls, d: dict) -> "Verdict":
        return cls(d["uid"], d["type"], d["passed"], d.get("reasoning", {}), d.get("error"))


def _as_bool(v) -> bool:
    if isinstance(v, bool):
        return v
    if isinstance(v, str):
        return v.strip().lower() == "true"
    return False


def parse_judge_output(parsed: dict) -> tuple[dict[str, bool], dict[str, str]]:
    passed, reasoning = {}, {}
    for d in DIMS:
        block = parsed[f"{d}_eval"]
        passed[d] = _as_bool(block["pass"])
        reasoning[d] = str(block.get("reasoning", ""))
    return passed, reasoning


class Judge:
    def __init__(self, gateway: Gateway):
        self.gw = gateway

    def judge(self, inst: Instance, responses: dict[str, str]) -> Verdict:
        resp = self.gw.chat([{"role": "user", "content": build_judge_user_prompt(inst, responses)}],
                            system=SYSTEM_PROMPT_ALL_IN_ONE_JUDGE, json_mode=True, tag="judge",
                            temperature=0)
        try:
            if not resp.ok_json:
                raise ValueError(resp.parse_error or "empty")
```

```
        passed, reasoning = parse_judge_output(resp.parsed)
        return Verdict(inst.uid, inst.type, passed, reasoning)
    except (KeyError, TypeError, ValueError) as e:
        return Verdict(inst.uid, inst.type, {d: False for d in DIMS}, error=f"{type(e).__name__}: {e}")
```

A.8 src/stale_repro/eval/stats.py (98 lines): Statistics

```
"""Scoring and statistics.

``Overall`` follows the paper: the unweighted mean of the six (type x dim) cell
accuracies. CIs are percentile bootstraps that resample *instances* within each
type (the three probes of one instance are not independent).
"""
from __future__ import annotations

from math import comb

import numpy as np

from ..data.loader import DIMS, DIM_NAMES, TYPE_NAMES
from .judge import Verdict

TYPES = ("T1", "T2")
CELLS = [(t, d) for t in TYPES for d in DIMS]


def _matrix(verdicts: list[Verdict], t: str) -> np.ndarray:
    rows = [[v.passed[d] for d in DIMS] for v in verdicts if v.type == t]
    return np.asarray(rows, dtype=float).reshape(-1, len(DIMS))


def cell_accuracies(verdicts: list[Verdict]) -> dict[str, float]:
    out: dict[str, float] = {}
    for t in TYPES:
        m = _matrix(verdicts, t)
        for j, d in enumerate(DIMS):
            out[f"{t}.{d}"] = float(m[:, j].mean()) if len(m) else float("nan")
    present = [v for v in out.values() if not np.isnan(v)]
    out["overall"] = float(np.mean(present)) if present else float("nan")
    return out


def bootstrap_ci(verdicts: list[Verdict], n_boot: int = 2000, alpha: float = 0.05,
                 seed: int = 0) -> dict[str, tuple[float, float]]:
    rng = np.random.default_rng(seed)
    mats = {t: _matrix(verdicts, t) for t in TYPES}
    samples: dict[str, list[float]] = {f"{t}.{d}": [] for t, d in CELLS} | {"overall": []}
    for _ in range(n_boot):
        cells = []
        for t in TYPES:
            m = mats[t]
            if not len(m):
                continue
            bm = m[rng.integers(0, len(m), len(m))].mean(axis=0)
            for j, d in enumerate(DIMS):
                samples[f"{t}.{d}"].append(bm[j])
                cells.append(bm[j])
        samples["overall"].append(float(np.mean(cells)))
    lo, hi = 100 * alpha / 2, 100 * (1 - alpha / 2)
    return {k: (float(np.percentile(v, lo)), float(np.percentile(v, hi))) for k, v in samples.items() if v}


def mcnemar_exact(a: list[bool], b: list[bool]) -> dict:
    """Exact two-sided McNemar test on paired binary outcomes."""
    if len(a) != len(b):
        raise ValueError("paired outcomes must have equal length")
    n01 = sum(1 for x, y in zip(a, b) if x and not y)
    n10 = sum(1 for x, y in zip(a, b) if y and not x)
    n = n01 + n10
    if n == 0:
        return {"a_only": 0, "b_only": 0, "p_value": 1.0}
    k = min(n01, n10)
    p = min(1.0, 2 * sum(comb(n, i) for i in range(k + 1)) / 2 ** n)
    return {"a_only": n01, "b_only": n10, "p_value": p}


def paired_outcomes(va: list[Verdict], vb: list[Verdict], dim: str | None = None,
                    type_: str | None = None) -> tuple[list[bool], list[bool]]:
    ia, ib = {v.uid: v for v in va}, {v.uid: v for v in vb}
    common = sorted(set(ia) & set(ib))
    dims = [dim] if dim else list(DIMS)
    a, b = [], []
    for u in common:
        if type_ and ia[u].type != type_:
            continue
        for d in dims:
            a.append(ia[u].passed[d]); b.append(ib[u].passed[d])
    return a, b


def results_table(verdicts: list[Verdict], ci: bool = True, **boot_kw) -> str:
    acc = cell_accuracies(verdicts)
    cis = bootstrap_ci(verdicts, **boot_kw) if ci else {}
```

```
head = ["n"] + [f"{TYPE_NAMES[t]} {DIM_NAMES[d]}" for t, d in CELLS] + ["Overall"]
keys = [f"{t}.{d}" for t, d in CELLS] + ["overall"]

def fmt(k: str) -> str:
    s = f"{100 * acc[k]:.1f}"
    if k in cis:
        s += f" [{100 * cis[k][0]:.1f}, {100 * cis[k][1]:.1f}]"
    return s

n_err = sum(1 for v in verdicts if v.error)
row = [f"{len(verdicts)}" + (f" ({n_err} judge err)" if n_err else "")] + [fmt(k) for k in keys]
return "| " + " | ".join(head) + " |\n|" + "---|" * len(head) + "\n| " + " | ".join(row) + " |"
```

A.9 src/stale_repro/runner.py (126 lines): Experiment runner

```
"""Experiment runner: config -> answers.jsonl -> verdicts.jsonl -> summary.md.

Resumable: instances already present in answers/verdicts are skipped, so a
crashed or interrupted run can simply be restarted with the same config.
"""
from __future__ import annotations

import json
import threading
from concurrent.futures import ThreadPoolExecutor, as_completed
from pathlib import Path
from typing import Callable

import yaml
from tqdm import tqdm

from .data.loader import DIMS, Instance, load_stale, select_subset
from .eval.judge import Judge, Verdict
from .eval.stats import bootstrap_ci, cell_accuracies, results_table
from .llm.gateway import Gateway, gateway_from_config
from .memory.base import MemorySystem


def _registry() -> dict[str, Callable[..., MemorySystem]]:
    from .memory.full_context import FullContext
    reg: dict[str, Callable[..., MemorySystem]] = {"full_context": FullContext}
    return reg


def _read_jsonl(path: Path) -> list[dict]:
    if not path.exists():
        return []
    with path.open(encoding="utf-8") as f:
        return [json.loads(line) for line in f if line.strip()]


class _JsonlWriter:
    def __init__(self, path: Path):
        self.path, self._lock = path, threading.Lock()

    def write(self, obj: dict) -> None:
        with self._lock, self.path.open("a", encoding="utf-8") as f:
            f.write(json.dumps(obj, ensure_ascii=False) + "\n")


def run_system_on_instance(system: MemorySystem, inst: Instance, session_mode: str = "full") -> dict:
    system.reset()
    system.begin_instance(inst)
    sessions = inst.sessions
    if session_mode == "relevant_only":  # oracle ablation, mirrors official CUPMem runner
        sessions = [inst.sessions[i] for i in inst.evidence_indices]
    system.ingest_many(sessions)
    answers, traces = {}, {}
    for d in DIMS:
        a = system.answer(inst.probes[d])
        answers[d], traces[d] = a.text, a.trace
    return {"uid": inst.uid, "type": inst.type, "answers": answers, "traces": traces}


def run(cfg: dict, out_root: str | Path = "runs", cache_dir: str | Path = "cache") -> dict:
    out = Path(out_root) / cfg["name"]
    out.mkdir(parents=True, exist_ok=True)
    (out / "config.yaml").write_text(yaml.safe_dump(cfg, sort_keys=False), encoding="utf-8")

    insts = select_subset(load_stale(cfg["data"]), **cfg.get("subset", {}))
    by_uid = {x.uid: x for x in insts}
    workers = int(cfg.get("workers", 4))
    session_mode = cfg.get("session_mode", "full")

    # ---- answer generation
    sys_cfg = cfg["system"]
    sys_gw = gateway_from_config(sys_cfg["llm"], cache_dir, log_path=out / "llm_calls.jsonl")
    factory = _registry()[sys_cfg["kind"]]
    ans_path = out / "answers.jsonl"
    done = {r["uid"] for r in _read_jsonl(ans_path)}
    todo = [x for x in insts if x.uid not in done]
    writer = _JsonlWriter(ans_path)

    def _answer(inst: Instance) -> None:
        system = factory(sys_gw, **sys_cfg.get("args", {}))
        writer.write(run_system_on_instance(system, inst, session_mode))

    _pool(_answer, todo, workers, "answering")
    answers = {r["uid"]: r for r in _read_jsonl(ans_path) if r["uid"] in by_uid}

    # ---- judging
```

```
    judge = Judge(gateway_from_config(cfg["judge"]["llm"], cache_dir, log_path=out / "judge_calls.jsonl"))
    ver_path = out / "verdicts.jsonl"
    judged = {r["uid"] for r in _read_jsonl(ver_path) if not r.get("error")}
    vwriter = _JsonlWriter(ver_path)
    to_judge = [by_uid[u] for u in answers if u not in judged]

    def _judge(inst: Instance) -> None:
        vwriter.write(judge.judge(inst, answers[inst.uid]["answers"]).to_dict())

    _pool(_judge, to_judge, workers, "judging")
    latest: dict[str, Verdict] = {}
    for r in _read_jsonl(ver_path):  # last verdict per uid wins (re-judged errors)
        if r["uid"] in by_uid:
            latest[r["uid"]] = Verdict.from_dict(r)
    verdicts = list(latest.values())

    summary = {
        "name": cfg["name"], "n_instances": len(verdicts),
        "judge_errors": sum(1 for v in verdicts if v.error),
        "accuracy": cell_accuracies(verdicts), "ci95": bootstrap_ci(verdicts),
        "usage_system": sys_gw.summary(), "usage_judge": judge.gw.summary(),
    }
    (out / "summary.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
    (out / "summary.md").write_text(f"# {cfg['name']}\n\n{results_table(verdicts)}\n", encoding="utf-8")
    return summary


def _pool(fn: Callable[[Instance], None], items: list[Instance], workers: int, desc: str) -> None:
    if not items:
        return
    errors: list[tuple[str, BaseException]] = []
    with ThreadPoolExecutor(max_workers=workers) as ex:
        futs = {ex.submit(fn, x): x.uid for x in items}
        for f in tqdm(as_completed(futs), total=len(futs), desc=desc):
            if (e := f.exception()) is not None:
                errors.append((futs[f], e))
    if errors:
        for uid, e in errors[:5]:
            print(f"[{desc}] {uid}: {type(e).__name__}: {e}")
        raise RuntimeError(f"{len(errors)} {desc} task(s) failed; rerun to resume")
```

A.10 scripts/run_experiment.py (35 lines): CLI

```
"""Usage: python scripts/run_experiment.py configs/b0_gpt4omini_dev.yaml [--n-per-type 5]"""
from __future__ import annotations

import argparse
import json

import yaml
from dotenv import load_dotenv

from stale_repro.runner import run


def main() -> None:
    p = argparse.ArgumentParser()
    p.add_argument("config")
    p.add_argument("--n-per-type", type=int, default=None, help="override subset size")
    p.add_argument("--workers", type=int, default=None)
    p.add_argument("--out-root", default="runs")
    p.add_argument("--cache-dir", default="cache")
    args = p.parse_args()

    load_dotenv()
    with open(args.config, encoding="utf-8") as f:
        cfg = yaml.safe_load(f)
    if args.n_per_type is not None:
        cfg.setdefault("subset", {})["n_per_type"] = args.n_per_type
        cfg["name"] = f"{cfg['name']}_n{args.n_per_type}"
    if args.workers is not None:
        cfg["workers"] = args.workers
    summary = run(cfg, args.out_root, args.cache_dir)
    print(json.dumps({k: summary[k] for k in ("name", "n_instances", "judge_errors", "accuracy")}, indent=2))


if __name__ == "__main__":
    main()
```

A.11 tests/conftest.py (86 lines): Test fixtures

```
"""Shared fixtures: a scripted fake LLM backend and hand-made toy scenarios."""
from __future__ import annotations

import json
from typing import Callable

import pytest

from stale_repro.data.loader import parse_record
from stale_repro.llm.gateway import Gateway, Usage


class FakeBackend:
    """Replies via a user-supplied function of (model, messages, params)."""

    def __init__(self, fn: Callable[[str, list[dict], dict], str]):
        self.fn, self.calls = fn, []

    def complete(self, model, messages, params):
        self.calls.append({"model": model, "messages": messages, "params": params})
        text = self.fn(model, messages, params)
        return text, Usage(prompt_tokens=sum(len(m["content"]) // 4 for m in messages),
                           completion_tokens=len(text) // 4)


def _session(*pairs: str) -> list[dict]:
    roles = ["user", "assistant"]
    return [{"role": roles[i % 2], "content": c} for i, c in enumerate(pairs)]


FILLER = _session("Any tips for a sourdough starter?", "Feed it daily with equal flour and water.")


def toy_record(uid: str, type_: str, old_msg: str, new_msg: str, m_old: str, m_new: str,
               explanation: str, queries: tuple[str, str, str], n_sessions: int = 6,
               idx_old: int = 1, idx_new: int = 4) -> dict:
    sessions = [FILLER for _ in range(n_sessions)]
    sessions[idx_old] = _session(old_msg, "Got it, thanks for sharing.")
    sessions[idx_new] = _session(new_msg, "Happy to help with that.")
    return {
        "uid": uid, "type": type_, "M_old": m_old, "M_new": m_new, "explanation": explanation,
        "probing_queries": {"dim1_query": queries[0], "dim2_query": queries[1], "dim3_query": queries[2]},
        "haystack_session": sessions,
        "timestamps": [f"2025-0{1 + i // 3}-{10 + i:02d} 10:00" for i in range(n_sessions)],
        "relevant_session_index": [idx_old, idx_new],
    }


SEATTLE = toy_record(
    "toy-seattle", "T1",
    "I've lived in Seattle for years and love the rain.",
    "Finally got the electricity and internet set up at my new apartment in Portland!",
    "User lives in Seattle.", "User set up utilities at a new apartment in Portland.",
    "location(city) is now Portland.",
    ("Does the user still live in Seattle?",
     "Since the user lives in Seattle, which Seattle farmers markets should they visit this weekend?",
     "Can you suggest a good weekend farmers market for me?"))

BROKEN_LEG = toy_record(
    "toy-leg", "T2",
    "I cycle to work every day, about 8km each way.",
    "The doctor put my leg in a cast for six weeks after the fracture.",
    "User cycles to work daily.", "User has a fractured leg in a cast for six weeks.",
    "A leg in a cast makes cycling to work impossible, so commute_mode is no longer cycling.",
    ("Does the user still cycle to work every day?",
     "Since the user bikes to work daily, what bike lights should they buy for dark mornings?",
     "How should I plan my commute for tomorrow?"))


@pytest.fixture
def toy_instances():
    return [parse_record(SEATTLE), parse_record(BROKEN_LEG)]


@pytest.fixture
def make_gateway(tmp_path):
    def _make(fn, **kw):
        backend = FakeBackend(fn)
        gw = Gateway(backend, model="fake-model", cache_path=tmp_path / "cache.sqlite",
                     log_path=tmp_path / "calls.jsonl", **kw)
        return gw, backend
    return _make


def judge_all_pass(model, messages, params) -> str:
    return json.dumps({f"dim{i}_eval": {"reasoning": "ok", "pass": True} for i in (1, 2, 3)})
```

A.12 tests/test_gateway.py (84 lines): Gateway tests

```
import json

import pytest

from stale_repro.llm.gateway import parse_json_loose


def test_cache_hit_skips_backend(make_gateway):
    gw, backend = make_gateway(lambda m, msgs, p: "hello")
    a = gw.chat("hi", tag="t")
    b = gw.chat("hi", tag="t")
    assert a.text == b.text == "hello"
    assert not a.cached and b.cached
    assert len(backend.calls) == 1
    s = gw.summary()["t"]
    assert s["calls"] == 2 and s["cached_calls"] == 1


def test_cache_key_depends_on_params(make_gateway):
    gw, backend = make_gateway(lambda m, msgs, p: "x")
    gw.chat("hi", temperature=0.0)
    gw.chat("hi", temperature=0.7)
    gw.chat("hi", model="other")
    assert len(backend.calls) == 3


def test_cache_persists_across_gateways(tmp_path, make_gateway):
    gw1, b1 = make_gateway(lambda m, msgs, p: "persisted")
    gw1.chat("q")
    gw2, b2 = make_gateway(lambda m, msgs, p: "SHOULD NOT BE CALLED")
    assert gw2.chat("q").text == "persisted"
    assert len(b2.calls) == 0


def test_json_mode_parses_fenced_output(make_gateway):
    gw, backend = make_gateway(lambda m, msgs, p: '```json\n{"a": 1}\n```')
    r = gw.chat("give json", json_mode=True)
    assert r.parsed == {"a": 1}
    assert backend.calls[0]["params"]["response_format"] == {"type": "json_object"}


def test_json_repair_retry_then_failure_is_counted(make_gateway):
    replies = iter(["not json at all", "still not json"])
    gw, backend = make_gateway(lambda m, msgs, p: next(replies))
    r = gw.chat("give json", json_mode=True, tag="x")
    assert not r.ok_json
    assert len(backend.calls) == 2
    assert gw.summary()["x"]["json_failures"] == 1


def test_json_repair_retry_succeeds(make_gateway):
    replies = iter(["oops", '{"ok": true}'])
    gw, _ = make_gateway(lambda m, msgs, p: next(replies))
    assert gw.chat("give json", json_mode=True).parsed == {"ok": True}


def test_json_schema_sets_response_format(make_gateway):
    gw, backend = make_gateway(lambda m, msgs, p: '{"label": "KEEP"}')
    schema = {"title": "adj", "type": "object", "properties": {"label": {"type": "string"}},
              "required": ["label"], "additionalProperties": False}
    gw.chat("x", json_schema=schema)
    rf = backend.calls[0]["params"]["response_format"]
    assert rf["type"] == "json_schema" and rf["json_schema"]["schema"] == schema


def test_usage_log_written(make_gateway, tmp_path):
    gw, _ = make_gateway(lambda m, msgs, p: "hi")
    gw.chat("a", tag="stage.one")
    lines = (tmp_path / "calls.jsonl").read_text().splitlines()
    assert json.loads(lines[0])["tag"] == "stage.one"


@pytest.mark.parametrize("text,expected", [
    ('{"a": 1}', {"a": 1}),
    ('Sure! Here it is: {"a": [1, 2]} hope that helps', {"a": [1, 2]}),
    ("```\n[1, 2]\n```", [1, 2]),
])
def test_parse_json_loose(text, expected):
    assert parse_json_loose(text) == expected


def test_parse_json_loose_raises():
    with pytest.raises(ValueError):
        parse_json_loose("no json here")
```

A.13 tests/test_pipeline.py (135 lines): Pipeline tests

```
import json
import random

import yaml

from stale_repro.data.loader import load_stale, select_subset
from stale_repro.eval.judge import Judge, build_judge_user_prompt
from stale_repro.eval.judge_prompts import SYSTEM_PROMPT_ALL_IN_ONE_JUDGE
from stale_repro.eval.stats import bootstrap_ci, cell_accuracies, mcnemar_exact
from stale_repro.eval.judge import Verdict
from stale_repro.memory.full_context import FullContext, format_haystack, trim_evidence_preserving
from stale_repro.runner import run_system_on_instance

from conftest import BROKEN_LEG, SEATTLE, judge_all_pass


def test_loader_roundtrip(tmp_path):
    p = tmp_path / "d.json"
    p.write_text(json.dumps([SEATTLE, BROKEN_LEG]))
    insts = load_stale(p)
    assert [x.type for x in insts] == ["T1", "T2"]
    s = insts[0]
    assert s.evidence_indices == (1, 4)
    assert "Seattle" in s.sessions[1].turns[0]["content"]
    assert s.probes["dim3"].is_task and not s.probes["dim2"].is_task


def test_subset_is_deterministic_and_stratified(toy_instances):
    many = []
    for i in range(10):
        for base in toy_instances:
            x = type(base)(**{**base.__dict__, "uid": f"{base.uid}-{i}"})
            many.append(x)
    a = select_subset(many, n_per_type=3, seed=1)
    b = select_subset(list(reversed(many)), n_per_type=3, seed=1)
    assert [x.uid for x in a] == [x.uid for x in b]
    assert sum(x.type == "T1" for x in a) == 3 and sum(x.type == "T2" for x in a) == 3


def test_format_haystack_matches_official():
    out = format_haystack([[{"role": "user", "content": "hi"}, {"role": "assistant", "content": "yo"}]],
                          ["2025-01-01 10:00"])
    assert out == "\n=== Session 1 [Time: 2025-01-01 10:00] ===\nUser: hi\nAssistant: yo\n"


def test_truncation_preserves_evidence(toy_instances):
    inst = toy_instances[0]
    big = [[{"role": "user", "content": "filler " * 400}] for _ in range(20)]
    big[3] = [{"role": "user", "content": "OLD_EVIDENCE"}]
    big[15] = [{"role": "user", "content": "NEW_EVIDENCE"}]
    ts = [f"t{i}" for i in range(20)]
    sessions, _, tokens, removed = trim_evidence_preserving(
        big, ts, 3, 15, inst.probes["dim1"], "gpt-4o-mini", 4000, random.Random(0))
    text = format_haystack(sessions)
    assert "OLD_EVIDENCE" in text and "NEW_EVIDENCE" in text
    assert removed > 0 and tokens <= 4000


def test_b0_sends_official_prompts(make_gateway, toy_instances):
    gw, backend = make_gateway(lambda m, msgs, p: "answer")
    rec = run_system_on_instance(FullContext(gw), toy_instances[1])
    assert set(rec["answers"]) == {"dim1", "dim2", "dim3"}
    sys_msgs = [c["messages"][0]["content"] for c in backend.calls]
    assert "accurately answer the question" in sys_msgs[0]
    assert "respond to the user's latest query directly" in sys_msgs[2]
    assert "[Latest Query]" in backend.calls[2]["messages"][1]["content"]
    assert "leg in a cast" in backend.calls[0]["messages"][1]["content"]


def test_relevant_only_mode_ingests_two_sessions(make_gateway, toy_instances):
    gw, backend = make_gateway(lambda m, msgs, p: "a")
    run_system_on_instance(FullContext(gw), toy_instances[0], session_mode="relevant_only")
    assert backend.calls[0]["messages"][1]["content"].count("=== Session") == 2


def test_judge_prompt_and_parse(make_gateway, toy_instances):
    gw, backend = make_gateway(judge_all_pass)
    v = Judge(gw).judge(toy_instances[0], {"dim1": "a", "dim2": "b", "dim3": "c"})
    assert v.passed == {"dim1": True, "dim2": True, "dim3": True} and v.error is None
    assert backend.calls[0]["messages"][0]["content"] == SYSTEM_PROMPT_ALL_IN_ONE_JUDGE
    assert backend.calls[0]["params"]["temperature"] == 0
    assert '- M_old: "User lives in Seattle."' in build_judge_user_prompt(
        toy_instances[0], {"dim1": "a", "dim2": "b", "dim3": "c"})


def test_judge_malformed_output_is_error_not_silent(make_gateway, toy_instances):
```

```
    gw, _ = make_gateway(lambda m, msgs, p: '{"dim1_eval": {"pass": true}}')
    v = Judge(gw).judge(toy_instances[0], {"dim1": "a", "dim2": "b", "dim3": "c"})
    assert v.error is not None and not any(v.passed.values())


def _v(uid, t, p1, p2, p3):
    return Verdict(uid, t, {"dim1": p1, "dim2": p2, "dim3": p3})


def test_overall_is_mean_of_six_cells():
    vs = [_v("a", "T1", True, False, True), _v("b", "T1", True, False, False),
          _v("c", "T2", False, False, True), _v("d", "T2", False, False, False)]
    acc = cell_accuracies(vs)
    assert acc["T1.dim1"] == 1.0 and acc["T1.dim3"] == 0.5 and acc["T2.dim3"] == 0.5
    assert abs(acc["overall"] - (1 + 0 + .5 + 0 + 0 + .5) / 6) < 1e-9
    ci = bootstrap_ci(vs, n_boot=200)
    assert ci["T1.dim1"] == (1.0, 1.0)


def test_mcnemar():
    assert mcnemar_exact([True] * 5, [True] * 5)["p_value"] == 1.0
    r = mcnemar_exact([True] * 10 + [False] * 10, [False] * 10 + [False] * 10)
    assert r["a_only"] == 10 and r["p_value"] < 0.01


def test_runner_end_to_end(tmp_path, monkeypatch, toy_instances):
    """Full config-driven run with a fake backend patched into the gateway factory."""
    import stale_repro.runner as runner
    from stale_repro.llm.gateway import Gateway
    from conftest import FakeBackend

    data = tmp_path / "d.json"
    data.write_text(json.dumps([SEATTLE, BROKEN_LEG]))

    def fake_factory(cfg, cache_dir, log_path=None):
        fn = judge_all_pass if cfg["model"] == "judge" else (lambda m, msgs, p: "sys answer")
        return Gateway(FakeBackend(fn), model=cfg["model"], cache_path=tmp_path / "c.sqlite", log_path=log_path)

    monkeypatch.setattr(runner, "gateway_from_config", fake_factory)
    cfg = {"name": "e2e", "data": str(data), "workers": 2,
           "system": {"kind": "full_context", "llm": {"model": "sys"}, "args": {}},
           "judge": {"llm": {"model": "judge"}}}
    s = runner.run(cfg, out_root=tmp_path / "runs")
    assert s["n_instances"] == 2 and s["accuracy"]["overall"] == 1.0
    # resume: nothing new to do, same result
    s2 = runner.run(cfg, out_root=tmp_path / "runs")
    assert s2["accuracy"] == s["accuracy"]
    assert (tmp_path / "runs" / "e2e" / "summary.md").exists()
    assert yaml.safe_load((tmp_path / "runs" / "e2e" / "config.yaml").read_text())["name"] == "e2e"
```

A.14 .gitignore (13 lines): Ignore rules

```
__pycache__/
*.pyc
*.egg-info/
.venv/
.env
.pytest_cache/

# large / third-party / generated
data/stale/*.json
external/
papers/*.pdf
runs/
cache/
```

A.15 .env.example (8 lines): API-key template

```
# Copy to .env and fill in. Never commit .env.
OPENAI_API_KEY=
# Judge (paper used Gemini-3.1-flash-lite via an OpenAI-compatible endpoint)
GEMINI_API_KEY=
# Hosted open-model provider (e.g. Together / OpenRouter / DeepInfra), for E2
OPEN_MODELS_API_KEY=
# Local vLLM server needs no key; set api_key_env to a var holding "EMPTY"
VLLM_API_KEY=EMPTY
```

Appendix B: Official judge prompt

System prompt, as copied into `src/stale_repro/eval/judge_prompts.py` from icedreamc/STALE @ ea7d391 (MIT). It is reproduced here without modification, including its original wording.

```
You are a strict and impartial expert evaluator for an AI Assistant Benchmark focusing on Long-Term Memory and Implici
    t Reasoning.

You will be provided with the Ground Truth Context of a user's state.
Then, you will evaluate THREE distinct responses generated by a Target AI Model to THREE different probing questions (
    Dimension 1, 2, and 3).

[Ground Truth Context]:
- M_old: The user's outdated state.
- M_new: The user's updated state.
- Hidden Logic: How M_new invalidates M_old.

You must grade each response strictly based on whether it correctly deduces and respects the hidden updated state.

### Scoring Rubrics (Boolean: true for Pass, false for Fail)

* **Dimension 1 (Explicit Probing):** Score `true` ONLY IF Response 1 awares that the information in M_old is possibly
     NO LONGER VALID. Score `false` if it assumes M_old is still true or claims ignorance.

* **Dimension 2 (Adversarial Robustness):**
  Question 2 contains a FALSE PREMISE based on M_old. Score `true` ONLY IF Response 2 detects the trap, or acknowledge
    s the new state in M_new. Score `false` if it blindly agrees with the false premise.

* **Dimension 3 (Implicit Task):**
  Score `true` ONLY IF the final action, plan, or recommendation in Response 3 is safely adheres to the NEW state esta
    blished by M_new.
  Score `false` if the action complies with the outdated M_old constraints, OR if the response is too generic, vague.

Evaluate strictly based on the underlying logic, not surface-level word matching.

### Output Format (JSON)
{
  "dim1_eval": {
    "reasoning": "Brief step-by-step reasoning against the Dim 1 rubric.",
    "pass": true/false
  },
  "dim2_eval": {
    "reasoning": "Brief step-by-step reasoning against the Dim 2 rubric.",
    "pass": true/false
  },
  "dim3_eval": {
    "reasoning": "Brief step-by-step reasoning against the Dim 3 rubric.",
    "pass": true/false
  }
}
```

User prompt template:

```
[Ground Truth Context]
- M_old: "{old_info}"
- M_new: "{M_new}"
- Hidden Logic: {explanation}

------------------------------------------------
[Dimension 1: Explicit Probing]
Question 1: {dim1_q}
Target Model Response 1: {dim1_a}

------------------------------------------------
[Dimension 2: Adversarial Robustness]
Question 2: {dim2_q}
Target Model Response 2: {dim2_a}

------------------------------------------------
[Dimension 3: Implicit Task]
Question 3: {dim3_q}
Target Model Response 3: {dim3_a}
```

Appendix C: Glossary

Term	Meaning
Implicit conflict	A later observation invalidates an earlier memory without explicitly negating it
Type I / Type II	Same-attribute conflict / conflict propagated from one attribute to another
SR / PR / IPA	State Resolution / Premise Resistance / Implicit Policy Adaptation (dim1 / dim2 / dim3)
Haystack	The 50-session conversation history, mostly distractor sessions from LongMemEval
Evidence sessions	S_old and S_new, the two sessions containing M_old and M_new
relevant_only	Official CUPMem mode that ingests only the two evidence sessions (oracle)
Evidence-preserving truncation	Dropping distractor sessions to fit a context window while keeping S_old and S_new
Adjudication	CUPMem's write-time decision about whether an old memory stays valid
B0 / B1 / B2 / B3 / M / M+	Full context / naive RAG / Mem0 / LightMem / our CUPMem / CUPMem with multi-hop propagation
Bootstrap CI	Confidence interval from resampling instances with replacement
McNemar test	Paired test for whether two systems differ on the same items
Gateway	Our single module through which every LLM call passes
vLLM	Open-source server that exposes open models through an OpenAI-compatible API